Process & Decision Documentation

GBDA 302: A2 Mid-Term Game Prototype

Group: Group 14A

Group Members:

Peter Kravets (21101536)

Min Htet Naing (21008098)

Nick Wang (21064962)

Joshua Metivier (21096515)

Fadeel Kosoko (21083665)

Project/Assignment Decisions

Description

For A2, our group created a playable p5.js prototype exploring the experience of panic disorder through interactive gameplay mechanics. The game places the player in a greenhouse environment where they must maintain multiple plants at the same time. Each plant gradually loses resources and must be actively maintained by providing water and light.

The core gameplay loop focuses on multitasking and time pressure. Players must constantly move between plants and decide which one needs attention first. If the player does not respond quickly enough, plants begin to deteriorate and may die. Because several plants can require attention simultaneously, the player experiences rising pressure and urgency.

This design was chosen to simulate the feeling of cognitive overload often associated with panic disorder. Rather than describing panic disorder through narrative text, the game attempts to communicate the experience through mechanics that require rapid decision-making and divided attention.

The objective of the game is to keep all plants alive for the duration of the level. If too many plants die, the system fails.

Key Decision/Change

One of the most important design decisions was to represent panic disorder through system management and rising cognitive load instead of relying on narrative explanation alone. Rather than telling the player what panic feels like, we wanted the player to experience a manageable but increasingly stressful situation through the mechanics themselves.

This led us to build the game around multitasking, uncertainty, and timed disruption. The greenhouse theme gave us a clear structure for balancing several responsibilities at once, while still keeping the game visually approachable and engaging. Managing water, light, and plant health creates a simple core loop, but surges and uncertainty layers make that loop more stressful over time.

Another key design decision involved the grounding mechanic. Initially, grounding was treated more like a mini-game. However, after showcase feedback, we recognized that this could make the mechanic feel stressful in itself, which contradicted its intended purpose. We therefore decided that grounding should instead function as a simpler and more calming system action that helps the player recover rather than adding another difficult challenge.

We also revised the onboarding structure after playtesting. Early testers found the opening instructions too long and the initial learning curve too abrupt. Because of this, we simplified the instruction screen and adjusted the opening phase so players could understand the basic management loop before more disruptive mechanics appeared.

Development Process

The development process followed the A2 pipeline of build → playtest → revise → showcase → finalize repository and documentation.

During the early development phase, the group focused on establishing the core gameplay loop. The priority was ensuring that the system of plant maintenance, resource management, and stabilization mechanics worked reliably and could be easily understood by players.

Once the basic mechanics were implemented, the group prepared a playable prototype for in-class playtesting. The prototype focused on communicating the main gameplay loop rather than visual polish. Controls, resource systems, and stabilization mechanics were prioritized so that players could understand the intended experience.

Playtesting observations were collected during the in-class testing session. The team focused on identifying moments where players appeared confused, overwhelmed, or uncertain about the objective of the game.

After playtesting, the group implemented several revisions before the mid-term showcase. These changes focused primarily on improving clarity and pacing rather than adding new mechanics.

Playtesting Findings and Iteration

Issue Observed During Playtesting	Change Implemented
The instruction screen contained too much text and was difficult to read quickly.	The instruction screen was simplified so the objective and controls were easier to understand.
The game introduced pressure too early for new players.	Early gameplay pacing was adjusted so players could learn the mechanics before experiencing higher pressure.

Some players were unsure of the main goal of the game.	The objective of keeping all plants alive was clarified in the instructions and onboarding.

These changes improved the onboarding experience and helped players understand the gameplay loop more quickly.

Showcase Feedback and Planned Improvements

Feedback from TA / Instructor	Planned Improvement
Panic disorder should be represented more clearly through cognitive load and triggers.	Adjust progression so stress and task complexity increase gradually throughout the level.
The grounding mechanic should feel calming rather than stressful.	Simplify the grounding interaction so it works as a recovery mechanic instead of a mini-game.
Instructions rely too heavily on text.	Reduce text and add clearer visual emphasis or icons for controls.
The game could benefit from clearer escalation and progression.	Improve pacing and add more structured increases in difficulty over time.

Role-Based Contributions

Team Member	Primary Role	Shadow Role	Main Contributions
Min Htet Naing	Project Coordinator (Gameplay Analysis / Affordances)	Verifier	Coordinated team workflow, tracked tasks and deadlines, reviewed gameplay affordances, and translated playtesting findings into concrete iteration plans.
Nick Wang	Structure + Documentation Lead (README / Submission)	Assistant Coordinator	Organized the README and documentation structure, ensured required sections were complete, and helped prepare the repository for submission.
Joshua Metivier	Build / QA + GitHub Pages Lead	Synthesizer	Built and tested the prototype, handled debugging and system stability, and ensured the game ran correctly on GitHub Pages in Chrome.
Peter Kravets	Playtest / Showcase Presenter + Lead Developer	Editor	Presented the game during playtesting and showcase sessions, supported core game development, and collected observations about player confusion, engagement, and frustration.
Fadeel Kosoko	Context Researcher (Representation + Global Framing)	Prompt Logger	Reviewed the representation of panic disorder, ensured respectful framing, and documented meaningful use of GenAI tools during the project.

GenAI Documentation (Used)

Date Used: Mar 1st - Mar 12th, 2026

Tool Disclosure: ChatGPT (GPT-5.2 Thinking)

Generative AI tools were used during the development of this project to support both game development and documentation tasks. The team used ChatGPT as a collaborative support tool for brainstorming mechanics, clarifying assignment requirements, refining written explanations, and assisting with parts of the programming process.

GenAI was used in several areas:

- Assisting with p5.js code generation and debugging during prototype development
- Helping structure the README and process documentation
- Summarizing playtesting and showcase feedback into clearer iteration notes
- Brainstorming possible game mechanics that represent panic disorder through gameplay systems

All AI-generated suggestions were reviewed, edited, and integrated by the team before being included in the project. The final game design, implementation decisions, and documentation were collaboratively verified by the group members.

GenAI was used as a support tool for development and iteration, while the overall project direction, design decisions, and final implementation remained under the control of the team.

Appendix

Prompts and answers:

Please modify my existing p5.js Garden Circuit code (do not rewrite from scratch). I'm going to provide the current code after this message. Implement the following changes carefully and keep everything else working (screens/states/scoring/surge/tension/etc.).
Assets note: I will place all image files inside `assets/` (a folder containing all game assets). Please load images from that folder (e.g., `assets/plant_good.png`).
1) Replace plant-bed squares with my plant images (health-based sprites)
Right now each plant bed is drawn as a colored rectangle. Replace that visual with images for the plant.
I have 4 images per plant state:
* `plant_good.png` (healthy)
* `plant_okay.png` (medium)
* `plant_bad.png` (low)
* `plant_dead.png` (wilted/dead)
Health thresholds (based on health 0–100)
* 67–100: good image
* 34–66: okay image
* 1–33: bad image
* 0: dead image
Requirements
* Use `preload()` to load all images from `assets/`.
* In each `PlantBed.draw()` (or wherever rendering happens), draw the correct image based on current `health`.
* Keep the existing UI overlays (meters/borders/selection highlight/uncertain zone shimmer/alerts) on top of the image so gameplay readability stays strong.
* If a plant is dead, it should also clearly look dead (dead image + optional desaturation overlay is fine).
* If an image fails to load, fall back gracefully to the old rectangle rendering (no crashes).

```
// ==============================================================
// GARDEN CIRCUIT — p5.js Prototype
// A real-time greenhouse management game with symbolic
// mechanics representing disruption, uncertainty, and coping.
// ==============================================================

// ==============================================================
// TUNING CONSTANTS — edit these to balance difficulty
// ==============================================================

const CANVAS_W = 1100;
const CANVAS_H = 750;

// Grid layout
const GRID_COLS = 3;
const GRID_ROWS = 3;
```

```
const TOTAL_BEDS = GRID_COLS * GRID_ROWS;

// Round / timing
const ROUND_DURATION = 90;
const GRACE_PERIOD = 15;

// Plant drain rates
const BASE_WATER_DRAIN = 5;
const BASE_LIGHT_DRAIN = 4;
const DRAIN_VARIATION = 1.5;
const HEALTH_LOSS_RATE = 7;
const HEALTH_RECOVERY_RATE = 2.5;
const CRITICAL_THRESHOLD = 25;
const HEALTHY_THRESHOLD = 50;

// Resource actions
const WATER_BOOST = 30;
const LIGHT_BOOST = 30;

// CHANGE #2 — Buffed airflow constants
const AIRFLOW_DURATION = 8;              // was 5 — lasts 60% longer
const AIRFLOW_DRAIN_REDUCTION = 0.3;     // was 0.5 — slows drain to 30% (stronger)
const AIRFLOW_HEALTH_RECOVERY = 1.5;     // NEW — heals 1.5 HP/sec while airflow active
const AIRFLOW_COOLDOWN = 3;              // was 4 — slightly shorter cooldown

const ACTION_COOLDOWN = 0.3;

// CHANGE #4 — Global action lock (anti-spam)
const ACTION_LOCK_DURATION = 0.2;        // NEW — 0.2s lockout after water/light

// Surge system
const SURGE_MIN_INTERVAL = 12;
```

```
const SURGE_MAX_INTERVAL = 25;
const SURGE_DURATION_MIN = 4;
const SURGE_DURATION_MAX = 7;
const SURGE_COOLDOWN_MIN = 8;
const FALSE_ALERT_COUNT = 3;
const SURGE_INPUT_DELAY = 0.1;
const SURGE_JITTER_AMOUNT = 3;

// Uncertain zones
const UNCERTAIN_ZONE_COUNT_MIN = 2;
const UNCERTAIN_ZONE_COUNT_MAX = 4;
const UNCERTAIN_DURATION_MIN = 8;
const UNCERTAIN_DURATION_MAX = 15;

// Tension meter
const TENSION_RISE_SURGE = 12;
const TENSION_RISE_UNCERTAIN = 4;
const TENSION_RISE_ALERTS = 1.2;
const TENSION_DECAY = 3;
const TENSION_OVERLOAD_RESET = 70;

// CHANGE #3 — More forgiving grounding constants
const GROUNDING_COOLDOWN = 10;
const GROUNDING_BEATS = 3;
const GROUNDING_BEAT_WINDOW = 0.45;      // was 0.3 — 50% wider timing window
const GROUNDING_BEAT_INTERVAL = 1.5;     // was 1.2 — slower rhythm, easier to read
const GROUNDING_SUCCESS_THRESHOLD = 1;   // NEW — only need 1/3 hits to succeed (was
hardcoded 2)
const GROUNDING_TENSION_REWARD = 30;
const GROUNDING_STABILITY_TIME = 5;
const GROUNDING_PARTIAL_REWARD = 10;
```

```
// CHANGE #1 — Plant image health thresholds
const PLANT_IMG_GOOD_MIN = 67;          // NEW — health >= 67 shows good image
const PLANT_IMG_OKAY_MIN = 34;          // NEW — health >= 34 shows okay image
const PLANT_IMG_BAD_MIN = 1;            // NEW — health >= 1 shows bad image
                                        // health <= 0 shows dead image

// Scoring
const SCORE_PER_HEALTHY_SEC = 1;
const SCORE_RESTORE_BONUS = 15;
const SCORE_GROUNDING_BONUS = 25;
const SCORE_SURGE_SURVIVE = 20;
const COMBO_THRESHOLD = 6;
const COMBO_MULTIPLIER_STEP = 0.1;
const COMBO_MAX_MULTIPLIER = 3.0;

// Win/Lose
const LOSE_WILTED_COUNT = 3;
const WIN_MIN_ALIVE = 7;

// Difficulty ramp
const DRAIN_RAMP_FACTOR = 0.5;
const SURGE_FREQ_RAMP = 0.4;

// Visual
const BED_MARGIN = 12;
const PANEL_WIDTH = 260;

// ==============================================================
// COLOR PALETTE
// ==============================================================
const COL = {
  bg:         [26, 30, 46],
```

```
  panelBg:      [20, 24, 38],
  bedNormal:    [40, 55, 45],
  bedSelected:  [60, 90, 70],
  bedUncertain: [55, 50, 65],
  bedDead:      [50, 35, 35],
  health:       [80, 200, 120],
  healthLow:    [200, 80, 80],
  water:        [80, 150, 220],
  light:        [230, 200, 80],
  airflow:      [180, 210, 230],
  tension:      [200, 100, 100],
  textPrimary:  [230, 235, 240],
  textSecondary: [160, 170, 180],
  surge:        [180, 60, 60],
  grounding:    [100, 180, 160],
  combo:        [240, 200, 80],
  accent:       [100, 200, 150],
  falseAlert:   [220, 130, 60],
  buttonBg:     [50, 70, 60],
  buttonHover:  [70, 100, 80],
  buttonText:   [230, 240, 230],
};

// ============================================================
// GAME STATE
// ============================================================
const STATE = {
  TITLE: 'title',
  INSTRUCTIONS: 'instructions',
  PLAYING: 'playing',
  GROUNDING: 'grounding',
  WIN: 'win',
```

```
  LOSE: 'lose',
  PAUSED: 'paused',
};

let gameState = STATE.TITLE;
let prevState = STATE.TITLE;

// Game variables
let beds = [];
let selectedBed = 0;
let timer = ROUND_DURATION;
let score = 0;
let tensionMeter = 0;
let comboCount = 0;
let comboMultiplier = 1.0;
let highestCombo = 0;

// Surge state
let surgeActive = false;
let surgeTimer = 0;
let surgeNextIn = 0;
let surgeCooldownLeft = 0;
let surgesCompleted = 0;
let surgeVisualIntensity = 0;

// Grounding state
let groundingAvailable = true;
let groundingCooldownLeft = 0;
let groundingSuccessCount = 0;
let groundingStabilityLeft = 0;

// Grounding mini-game state
```

```
let groundingBeats = [];
let groundingBeatIndex = 0;
let groundingPhase = 0;
let groundingCircleRadius = 0;
let groundingHits = 0;
let groundingTimer = 0;
let groundingFeedback = '';
let groundingFeedbackTimer = 0;
let groundingComplete = false;

// Action cooldowns
let waterCooldown = 0;
let lightCooldown = 0;
let airflowCooldown = 0;

// CHANGE #4 — Global action lock timer
let actionLockTimer = 0;

// Input delay queue (for surge effect)
let inputQueue = [];
let surgeJitterX = 0;
let surgeJitterY = 0;

// Particles
let particles = [];
const MAX_PARTICLES = 60;

// Stats for end screen
let stats = {
  surgesSurvived: 0,
  groundingAttempts: 0,
  groundingSuccesses: 0,
```

```
  plantsWilted: 0,
  peakCombo: 0,
  totalRestores: 0,
};

let titlePulses = [];
let buttons = [];
let difficultyMult = 1.0;
let reducedEffects = false;

// ============================================================
// CHANGE #1 — Plant image variables
// ============================================================
let plantGoodImg = null;
let plantOkayImg = null;
let plantBadImg = null;
let plantDeadImg = null;
let imagesLoaded = false;  // fallback flag if images fail

// ============================================================
// CHANGE #1 — Preload plant images from assets/ folder
// ============================================================
function preload() {
  // Attempt to load all 4 plant state images.
  // If any fail, imagesLoaded stays false and we fall back
  // to the old rectangle rendering (no crash).
  try {
    plantGoodImg = loadImage('assets/plant_good.png',
      () => {}, // success callback (no-op)
      () => { plantGoodImg = null; } // failure callback
    );
    plantOkayImg = loadImage('assets/plant_okay.png',
```

```
    () => {},
    () => { plantOkayImg = null; }
  );
  plantBadImg = loadImage('assets/plant_bad.png',
    () => {},
    () => { plantBadImg = null; }
  );
  plantDeadImg = loadImage('assets/plant_dead.png',
    () => {},
    () => { plantDeadImg = null; }
  );
} catch (e) {
  console.warn('Plant images could not be loaded, using fallback rendering.', e);
}
}

// Helper: check after setup whether all images loaded successfully
function checkImagesLoaded() {
  imagesLoaded = (plantGoodImg !== null && plantOkayImg !== null &&
                  plantBadImg !== null && plantDeadImg !== null);
  if (!imagesLoaded) {
    console.warn('One or more plant images missing — using fallback shapes.');
  }
}

// Helper: get the correct image for a given health value
function getPlantImage(health) {
  if (health <= 0) return plantDeadImg;
  if (health < PLANT_IMG_OKAY_MIN) return plantBadImg;
  if (health < PLANT_IMG_GOOD_MIN) return plantOkayImg;
  return plantGoodImg;
}
```

```
// ==============================================================
// PLANT BED CLASS
// ==============================================================
class PlantBed {
  constructor(col, row, index) {
    this.col = col;
    this.row = row;
    this.index = index;

    this.health = 80 + random(-10, 10);
    this.water = 60 + random(-15, 15);
    this.light = 60 + random(-15, 15);

    this.drainRateWater = BASE_WATER_DRAIN + random(-DRAIN_VARIATION, DRAIN_VARIATION);
    this.drainRateLight = BASE_LIGHT_DRAIN + random(-DRAIN_VARIATION, DRAIN_VARIATION);

    this.airflowActive = false;
    this.airflowTimer = 0;

    this.isUncertainZone = false;
    this.uncertainTimer = 0;

    this.hasFalseAlert = false;
    this.isWilted = false;

    this.x = 0;
    this.y = 0;
    this.w = 0;
    this.h = 0;

    this.flashTimer = 0;
```

```
  this.restoreFlash = 0;
  this.wasStressed = false;
}

get trueUrgency() {
  if (this.health <= 0) return 'dead';
  if (this.water < CRITICAL_THRESHOLD || this.light < CRITICAL_THRESHOLD) return 'critical';
  if (this.water < HEALTHY_THRESHOLD || this.light < HEALTHY_THRESHOLD) return 'warning';
  return 'healthy';
}

get displayUrgency() {
  if (this.hasFalseAlert) return 'critical';
  if (this.isUncertainZone && surgeActive) {
    if (random() < 0.2) {
      return random() < 0.5 ? 'warning' : 'healthy';
    }
  }
  return this.trueUrgency;
}

update(dt) {
  if (this.isWilted) return;

  let drainMult = difficultyMult;
  if (this.airflowActive) drainMult *= AIRFLOW_DRAIN_REDUCTION;

  this.water -= this.drainRateWater * drainMult * dt;
  this.light -= this.drainRateLight * drainMult * dt;
  this.water = constrain(this.water, 0, 100);
  this.light = constrain(this.light, 0, 100);
```

```
  if (this.water < CRITICAL_THRESHOLD || this.light < CRITICAL_THRESHOLD) {
    // CHANGE #2 — Airflow reduces health loss when plant is stressed
    let healthLoss = HEALTH_LOSS_RATE;
    if (this.airflowActive) healthLoss *= 0.5; // halve health loss during airflow
    this.health -= healthLoss * dt;
    this.wasStressed = true;
  } else if (this.water >= HEALTHY_THRESHOLD && this.light >= HEALTHY_THRESHOLD) {
    this.health += HEALTH_RECOVERY_RATE * dt;
    if (this.wasStressed && this.health > 50) {
      score += SCORE_RESTORE_BONUS * comboMultiplier;
      stats.totalRestores++;
      this.restoreFlash = 0.5;
      this.wasStressed = false;
      spawnParticle(this.x + this.w / 2, this.y + this.h / 2, COL.accent, '+' +
floor(SCORE_RESTORE_BONUS * comboMultiplier));
    }
  }

  // CHANGE #2 — Airflow grants passive health recovery while active
  if (this.airflowActive && this.health > 0 && this.health < 100) {
    this.health += AIRFLOW_HEALTH_RECOVERY * dt;
  }

  this.health = constrain(this.health, 0, 100);

  if (this.health <= 0) {
    this.isWilted = true;
    stats.plantsWilted++;
  }

  if (this.airflowActive) {
    this.airflowTimer -= dt;
```

```
    if (this.airflowTimer <= 0) {
      this.airflowActive = false;
    }
  }

  if (this.isUncertainZone) {
    this.uncertainTimer -= dt;
    if (this.uncertainTimer <= 0) {
      this.isUncertainZone = false;
    }
  }

  if (this.flashTimer > 0) this.flashTimer -= dt;
  if (this.restoreFlash > 0) this.restoreFlash -= dt;
}

applyWater() {
  if (this.isWilted) return;
  this.water = constrain(this.water + WATER_BOOST, 0, 100);
  this.flashTimer = 0.3;
}

applyLight() {
  if (this.isWilted) return;
  this.light = constrain(this.light + LIGHT_BOOST, 0, 100);
  this.flashTimer = 0.3;
}

applyAirflow() {
  if (this.isWilted) return;
  this.airflowActive = true;
  this.airflowTimer = AIRFLOW_DURATION;
```

```
  }
}

// ==============================================================
// PARTICLE SYSTEM
// ==============================================================
function spawnParticle(x, y, col, txt) {
  if (particles.length >= MAX_PARTICLES) particles.shift();
  particles.push({
    x, y, col: [...col],
    txt: txt || '',
    life: 1.0,
    vy: -40,
  });
}

function updateParticles(dt) {
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.life -= dt * 0.8;
    p.y += p.vy * dt;
    if (p.life <= 0) particles.splice(i, 1);
  }
}

function drawParticles() {
  for (let p of particles) {
    let a = p.life * 255;
    fill(p.col[0], p.col[1], p.col[2], a);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(14);
```

```
    textStyle(BOLD);
    text(p.txt, p.x, p.y);
  }
}

// ==============================================================
// BUTTON HELPER
// ==============================================================
class Button {
  constructor(x, y, w, h, label, callback, id) {
    this.x = x;
    this.y = y;
    this.w = w;
    this.h = h;
    this.label = label;
    this.callback = callback;
    this.id = id || label;
    this.hovered = false;
    this.visible = true;
  }

  contains(px, py) {
    return px >= this.x && px <= this.x + this.w &&
           py >= this.y && py <= this.y + this.h;
  }

  draw() {
    if (!this.visible) return;
    let col = this.hovered ? COL.buttonHover : COL.buttonBg;
    fill(col[0], col[1], col[2]);
    stroke(COL.accent[0], COL.accent[1], COL.accent[2], 120);
    strokeWeight(1.5);
```

```
    rect(this.x, this.y, this.w, this.h, 6);

    fill(COL.buttonText[0], COL.buttonText[1], COL.buttonText[2]);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(16);
    textStyle(BOLD);
    text(this.label, this.x + this.w / 2, this.y + this.h / 2);
  }

  checkHover(mx, my) {
    this.hovered = this.visible && this.contains(mx, my);
  }

  checkClick(mx, my) {
    if (this.visible && this.contains(mx, my)) {
      this.callback();
      return true;
    }
    return false;
  }
}

// ============================================================
// LAYOUT CALCULATIONS
// ============================================================
let boardX, boardY, boardW, boardH;
let bedW, bedH;
let panelX;

function computeLayout() {
  panelX = CANVAS_W - PANEL_WIDTH;
```

```
  boardX = 20;
  boardY = 20;
  boardW = panelX - 40;
  boardH = CANVAS_H - 40;

  bedW = (boardW - BED_MARGIN * (GRID_COLS + 1)) / GRID_COLS;
  bedH = (boardH - BED_MARGIN * (GRID_ROWS + 1)) / GRID_ROWS;

  for (let bed of beds) {
    bed.x = boardX + BED_MARGIN + bed.col * (bedW + BED_MARGIN);
    bed.y = boardY + BED_MARGIN + bed.row * (bedH + BED_MARGIN);
    bed.w = bedW;
    bed.h = bedH;
  }
}

// ============================================================
// INITIALIZATION
// ============================================================
function initGame() {
  beds = [];
  for (let r = 0; r < GRID_ROWS; r++) {
    for (let c = 0; c < GRID_COLS; c++) {
      beds.push(new PlantBed(c, r, r * GRID_COLS + c));
    }
  }
  computeLayout();

  selectedBed = 0;
  timer = ROUND_DURATION;
  score = 0;
  tensionMeter = 0;
```

```
comboCount = 0;
comboMultiplier = 1.0;
highestCombo = 0;
difficultyMult = 1.0;

surgeActive = false;
surgeTimer = 0;
surgeNextIn = GRACE_PERIOD + random(3, 8);
surgeCooldownLeft = 0;
surgesCompleted = 0;
surgeVisualIntensity = 0;

groundingAvailable = true;
groundingCooldownLeft = 0;
groundingSuccessCount = 0;
groundingStabilityLeft = 0;

waterCooldown = 0;
lightCooldown = 0;
airflowCooldown = 0;
actionLockTimer = 0;  // CHANGE #4 — reset action lock

inputQueue = [];
particles = [];

stats = {
  surgesSurvived: 0,
  groundingAttempts: 0,
  groundingSuccesses: 0,
  plantsWilted: 0,
  peakCombo: 0,
  totalRestores: 0,
```

```
  };
}

// ==============================================================
// SURGE SYSTEM
// ==============================================================
function startSurge() {
  surgeActive = true;
  surgeTimer = random(SURGE_DURATION_MIN, SURGE_DURATION_MAX);
  surgeVisualIntensity = 1.0;

  let available = beds.filter(b => !b.isWilted && b.trueUrgency !== 'critical');
  shuffle(available, true);
  let count = min(FALSE_ALERT_COUNT, available.length);
  for (let i = 0; i < count; i++) {
    available[i].hasFalseAlert = true;
  }
}

function endSurge() {
  surgeActive = false;
  surgesCompleted++;
  stats.surgesSurvived++;
  score += SCORE_SURGE_SURVIVE * comboMultiplier;

  for (let b of beds) b.hasFalseAlert = false;

  let available = beds.filter(b => !b.isWilted);
  shuffle(available, true);
  let count = floor(random(UNCERTAIN_ZONE_COUNT_MIN, UNCERTAIN_ZONE_COUNT_MAX + 1));
  count = min(count, available.length);
  for (let i = 0; i < count; i++) {
```

```
  available[i].isUncertainZone = true;
  available[i].uncertainTimer = random(UNCERTAIN_DURATION_MIN,
UNCERTAIN_DURATION_MAX);
 }

 let interval = lerp(SURGE_MAX_INTERVAL, SURGE_MIN_INTERVAL, (ROUND_DURATION - timer) /
ROUND_DURATION * SURGE_FREQ_RAMP);
 surgeNextIn = random(interval * 0.8, interval * 1.2);
 surgeCooldownLeft = SURGE_COOLDOWN_MIN;
}

function updateSurge(dt) {
 if (surgeActive) {
  surgeTimer -= dt;
  tensionMeter += TENSION_RISE_SURGE * dt;

  if (!reducedEffects) {
   surgeJitterX = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
   surgeJitterY = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
  }

  if (surgeTimer <= 0) {
   endSurge();
  }
 } else {
  surgeJitterX = lerp(surgeJitterX, 0, 0.2);
  surgeJitterY = lerp(surgeJitterY, 0, 0.2);

  surgeCooldownLeft -= dt;
  surgeNextIn -= dt;
```

```
  if (surgeNextIn <= 0 && surgeCooldownLeft <= 0 && timer < (ROUND_DURATION -
GRACE_PERIOD)) {
    startSurge();
  }
}

surgeVisualIntensity = surgeActive ? lerp(surgeVisualIntensity, 1, 0.1) : lerp(surgeVisualIntensity, 0,
0.05);
}

// ============================================================
// GROUNDING MINI-GAME
// ============================================================
function startGrounding() {
  // CHANGE #4 — Respect action lock
  if (actionLockTimer > 0) return;
  if (!groundingAvailable || groundingCooldownLeft > 0) return;

  prevState = gameState;
  gameState = STATE.GROUNDING;
  stats.groundingAttempts++;

  groundingBeatIndex = 0;
  groundingHits = 0;
  groundingTimer = 0;
  groundingComplete = false;
  groundingCircleRadius = 0;
  groundingFeedback = '';
  groundingFeedbackTimer = 0;

  groundingBeats = [];
  for (let i = 0; i < GROUNDING_BEATS; i++) {
```

```
  groundingBeats.push({
    time: 1.0 + i * GROUNDING_BEAT_INTERVAL,
    hit: false,
    result: '',
  });
}
}

function updateGrounding(dt) {
  groundingTimer += dt;

  // CHANGE #3 — Slower pulse rhythm using GROUNDING_BEAT_INTERVAL
  let beatPhase = (groundingTimer % GROUNDING_BEAT_INTERVAL) /
GROUNDING_BEAT_INTERVAL;
  groundingCircleRadius = 30 + sin(beatPhase * PI) * 50;

  if (groundingFeedbackTimer > 0) groundingFeedbackTimer -= dt;

  if (groundingBeatIndex >= GROUNDING_BEATS) {
    if (!groundingComplete) {
      groundingComplete = true;
      groundingTimer = 0;
    }
  }

  if (groundingComplete) {
    if (groundingTimer > 1.5) {
      finishGrounding();
    }
  } else {
    if (groundingBeatIndex < GROUNDING_BEATS) {
      let beat = groundingBeats[groundingBeatIndex];
```

```
    // CHANGE #3 — Use wider window constant for miss detection
    if (groundingTimer > beat.time + GROUNDING_BEAT_WINDOW + 0.15) {
      beat.result = 'missed';
      groundingFeedback = 'Missed';
      groundingFeedbackTimer = 0.6;
      groundingBeatIndex++;
    }
  }
}

// Grounding fully pauses plant decay (player isn't punished for coping)
// (no bed.update call here)
}

function groundingInput() {
  if (groundingComplete || groundingBeatIndex >= GROUNDING_BEATS) return;

  let beat = groundingBeats[groundingBeatIndex];
  let diff = abs(groundingTimer - beat.time);

  // CHANGE #3 — Uses wider GROUNDING_BEAT_WINDOW for easier hits
  if (diff <= GROUNDING_BEAT_WINDOW) {
    beat.hit = true;
    beat.result = 'hit';
    groundingHits++;
    // CHANGE #3 — Extra positive feedback for timing quality
    if (diff <= GROUNDING_BEAT_WINDOW * 0.4) {
      groundingFeedback = 'Perfect!';
    } else {
      groundingFeedback = 'Steady!';
    }
    groundingFeedbackTimer = 0.5;
```

```
} else if (groundingTimer < beat.time - GROUNDING_BEAT_WINDOW) {
  beat.result = 'early';
  groundingFeedback = 'Too Early';
  groundingFeedbackTimer = 0.5;
} else {
  beat.result = 'late';
  groundingFeedback = 'Too Late';
  groundingFeedbackTimer = 0.5;
}
groundingBeatIndex++;
}

function finishGrounding() {
// CHANGE #3 — Use configurable success threshold (default 1/3 hits)
let success = groundingHits >= GROUNDING_SUCCESS_THRESHOLD;

if (success) {
  tensionMeter = max(0, tensionMeter - GROUNDING_TENSION_REWARD);
  groundingStabilityLeft = GROUNDING_STABILITY_TIME;
  groundingSuccessCount++;
  stats.groundingSuccesses++;
  score += SCORE_GROUNDING_BONUS * comboMultiplier;

  for (let b of beds) b.hasFalseAlert = false;
} else {
  tensionMeter = max(0, tensionMeter - GROUNDING_PARTIAL_REWARD);
}

groundingCooldownLeft = GROUNDING_COOLDOWN;
gameState = STATE.PLAYING;
}
```

```
// ==============================================================
// TENSION METER
// ==============================================================
function updateTension(dt) {
  let activeAlerts = beds.filter(b => !b.isWilted && b.trueUrgency === 'critical').length;
  tensionMeter += activeAlerts * TENSION_RISE_ALERTS * dt * 0.3;

  if (!surgeActive && activeAlerts < 2) {
    tensionMeter -= TENSION_DECAY * dt;
  }

  if (groundingStabilityLeft > 0) {
    tensionMeter -= TENSION_DECAY * 2 * dt;
    groundingStabilityLeft -= dt;
  }

  tensionMeter = constrain(tensionMeter, 0, 100);

  if (tensionMeter >= 100) {
    tensionMeter = TENSION_OVERLOAD_RESET;
    comboMultiplier = max(1.0, comboMultiplier - 0.5);
  }
}

// ==============================================================
// COMBO SYSTEM
// ==============================================================
function updateCombo(dt) {
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40 && b.water > 30 && b.light > 30).length;

  if (healthyCount >= COMBO_THRESHOLD) {
    comboCount++;
```

```
    comboMultiplier = min(COMBO_MAX_MULTIPLIER, 1.0 + floor(comboCount / 60) *
COMBO_MULTIPLIER_STEP);
  } else {
    if (comboCount > 0) {
      comboCount = max(0, comboCount - 3);
      if (comboCount === 0) comboMultiplier = 1.0;
    }
  }

  highestCombo = max(highestCombo, comboMultiplier);
  stats.peakCombo = highestCombo;
}

function updateScore(dt) {
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40).length;
  score += healthyCount * SCORE_PER_HEALTHY_SEC * comboMultiplier * dt;
}

function updateDifficulty() {
  let elapsed = ROUND_DURATION - timer;
  let progress = elapsed / ROUND_DURATION;
  difficultyMult = 1.0 + progress * DRAIN_RAMP_FACTOR;
}

// ============================================================
// ACTION HANDLING
// ============================================================
function doAction(type) {
  // CHANGE #4 — Block all actions during action lock
  if (actionLockTimer > 0) return;

  let bed = beds[selectedBed];
```

```
  if (!bed || bed.isWilted) return;

  if (bed.isUncertainZone) {
    tensionMeter += TENSION_RISE_UNCERTAIN;
  }

  if (type === 'water' && waterCooldown <= 0) {
    bed.applyWater();
    waterCooldown = ACTION_COOLDOWN;
    actionLockTimer = ACTION_LOCK_DURATION;  // CHANGE #4 — trigger global lock
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.water, 'Water');
  } else if (type === 'light' && lightCooldown <= 0) {
    bed.applyLight();
    lightCooldown = ACTION_COOLDOWN;
    actionLockTimer = ACTION_LOCK_DURATION;  // CHANGE #4 — trigger global lock
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.light, 'Light');
  } else if (type === 'airflow' && airflowCooldown <= 0) {
    bed.applyAirflow();
    airflowCooldown = AIRFLOW_COOLDOWN;
    // CHANGE #4 — No action lock for airflow (already has long cooldown)
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.airflow, 'Airflow');
  }
}

function processInputQueue(dt) {
  for (let i = inputQueue.length - 1; i >= 0; i--) {
    inputQueue[i].delay -= dt;
    if (inputQueue[i].delay <= 0) {
      inputQueue[i].action();
      inputQueue.splice(i, 1);
    }
  }
```

```
}

function queueAction(fn) {
  if (surgeActive) {
    inputQueue.push({ action: fn, delay: SURGE_INPUT_DELAY });
  } else {
    fn();
  }
}

// ==============================================================
// CHECK WIN / LOSE
// ==============================================================
function checkEndConditions() {
  let wiltedCount = beds.filter(b => b.isWilted).length;

  if (wiltedCount >= LOSE_WILTED_COUNT) {
    gameState = STATE.LOSE;
    return;
  }

  if (timer <= 0) {
    let aliveCount = beds.filter(b => !b.isWilted).length;
    if (aliveCount >= WIN_MIN_ALIVE) {
      gameState = STATE.WIN;
    } else {
      gameState = STATE.LOSE;
    }
  }
}

function computeGrade() {
```

```
  let maxPossible = TOTAL_BEDS * SCORE_PER_HEALTHY_SEC * ROUND_DURATION * 2;
  let ratio = score / maxPossible;
  if (ratio > 0.6) return 'A';
  if (ratio > 0.4) return 'B';
  if (ratio > 0.25) return 'C';
  return 'D';
}

function getBedAtMouse(mx, my) {
  for (let i = 0; i < beds.length; i++) {
    let b = beds[i];
    if (mx >= b.x && mx <= b.x + b.w && my >= b.y && my <= b.y + b.h) {
      return i;
    }
  }
  return -1;
}
```

Prompt:

Great progress — a couple of important fixes/tuning changes:

1) Instructions screen layout bug

On the **Instructions / How to Play** screen, the **"Back to Title"** button is overlapping the bottom text (the grounding section bullets).
Please **move the button so it does not cover any text**.

Preferred fix:

- Place the button **below all instruction text** with clear spacing, OR
- Move it to the **bottom-right corner** with enough margin, and ensure the text area stops above it.
  Please make sure **all instruction text is fully visible and readable** on the 1100x750 canvas.
  ### 2) Gameplay difficulty tuning (too hard right now)
  Please make the game **easier to manage** by slowing depletion rates.
  Specifically:
- Make **Water and Light deplete slower** for all beds (reduce base drain rates noticeably)
- Keep the game challenging, but less overwhelming in the first 30–45 seconds
- If needed, slightly reduce:
  - number of simultaneous critical alerts
  - how fast multiple beds spiral at once

  Goal: it should feel tense but manageable for first-time players in a class demo.
  ### Suggested tuning direction (if helpful)
- Reduce base water/light drain by ~25–40%
- Slightly slower difficulty ramp over time
- Keep the same mechanics and structure, just improve playability/balance

Please update the code with these changes. (just place the sections that

```
// FIND AND REPLACE THESE VALUES:

// Plant drain rates — REDUCED for easier gameplay

const BASE_WATER_DRAIN = 3.2;      // was 5.0

const BASE_LIGHT_DRAIN = 2.8;      // was 4.5

const DRAIN_VARIATION = 0.8;       // was 1.5 — less random spread

const HEALTH_LOSS_RATE = 5.5;      // was 8.0 — slower health decay

const HEALTH_RECOVERY_RATE = 3.0;   // was 2.0 — faster recovery

// Resource actions — slightly more generous

const WATER_BOOST = 35;            // was 30

const LIGHT_BOOST = 35;            // was 30
```

```
// Difficulty ramp — gentler curve

const DRAIN_RAMP_FACTOR = 0.35;   // was 0.6

const SURGE_FREQ_RAMP = 0.25;     // was 0.4

// Surge timing — more breathing room

const SURGE_MIN_INTERVAL = 16;    // was 12

const SURGE_MAX_INTERVAL = 30;    // was 25

const GRACE_PERIOD = 20;          // was 15

// Tension — slower buildup

const TENSION_RISE_SURGE = 8;     // was 12

const TENSION_RISE_UNCERTAIN = 4;  // was 6

const TENSION_RISE_ALERTS = 1.2;   // was 2
```

Prompt:

> the ai will do all the other work simply write a prompt for these comands in detail, you dont need exact numbers just directions"with that i made this image, i want to change the start screen to this, im showing it as an image, but it will be a video mp4 just this but animated, make it so with the canvas size of this image you can find the buttons location on the image, and place a square over those parts of the image so when you click it it brings you to that part of the game, but if you press instuctions make it so it just opens a tab with the instructions overtop of the video but you can just see the border of the openeing screen so it doesnt fully take you out from that experience, also in game make it so esc also pauses and make it so you can go back to main menu from pause menu (give prompt)"

Please modify my existing p5.js game (Garden Circuit) to update the **title screen UI/flow** and **pause behavior** as follows. I will provide the current code and the new title media files after this message. Do not rewrite the whole project—apply focused edits.

1) Replace the Start Screen background with my animated MP4 (title video)

- I currently have a static title screen. Replace it so the title screen displays an **MP4 video** (the animated version of the provided image) as the full background.
- Load the MP4 from assets/ (e.g., assets/title.mp4) and draw it behind any UI.
- The title background should loop smoothly and autoplay when entering the title screen.
- Keep performance reasonable (don't spawn multiple video instances; create it once and reuse it).

2) Use the title media's canvas size + approximate button regions (invisible clickable rectangles)

- Set the canvas size to match the title media's aspect ratio (or letterbox it neatly). The goal is that the video/image lines up with the intended button positions.
- The title screen has two visual buttons in the art: **"Start Game"** and **"Instructions"**.
- Create two **invisible hitboxes** (simple rectangles) placed over those button areas.
    - You do NOT need exact pixel-perfect values, but you should:
        - estimate the location based on the image/video layout
        - keep them large enough for easy clicking
        - optionally add a debug toggle key (like H) to show the rectangles for testing
- When the player clicks inside the Start hitbox, start the game normally.

3) Instructions should open as an overlay panel on top of the title video (not a separate screen)

- Clicking the Instructions hitbox should NOT transition away from the title screen.
- Instead, open an **instructions overlay "panel/tab"** on top of the still-playing title video:

    - The video remains visible in the background.
    - The overlay should be centered and readable, with a semi-transparent dark backing.
    - The overlay should leave some of the title border/edges visible so it still feels like the title screen experience (do not cover the entire screen with an opaque full-screen page).
- Include a clear “Close” button (or key like Esc/I) to close the overlay and return to the title screen state (video continues).
- While the overlay is open, prevent clicks from triggering the Start button underneath (overlay captures input).

4) Add ESC as a pause toggle during gameplay

- During gameplay, pressing **Esc** should pause/resume (same as P if P already exists).
- Ensure it does not conflict with closing the instructions overlay on the title screen (context-sensitive behavior is fine: Esc closes overlay on title; Esc pauses in gameplay).

5) Pause menu should include “Back to Main Menu”

- When paused, show a pause overlay/menu with:
    - Resume
    - (Optional) Restart
    - **Back to Main Menu**
- Back to Main Menu should:
    - stop gameplay loop properly
    - reset round state/variables as needed

    - return to the title screen with the MP4 video playing
- Make sure returning to title does not create duplicate videos or leak resources.

Implementation guidance (no exact numbers required)

- Use rectangles/hitboxes for Start/Instructions based on the **visual button locations** in the title art.
- Keep code organized:
    - a STATE_TITLE with sub-state showInstructionsOverlay boolean is fine
    - a reusable drawTitle() that draws video + hitboxes + overlay
- If video is hard to draw directly in p5:
    - create the video element with createVideo(), hide it, and draw it with image(video, 0,0,w,h) each frame.
- Put new constants at the top for:
    - video filename
    - hitbox positions/sizes (approx)
    - overlay layout sizes

After changes, briefly explain:

- what files were edited
- which constants I can tweak to reposition hitboxes and resize the overlay.

make the border like big trees, change the angle and make the format different give it a new look entierly, bring start game and make instructions a button aswell

make an 1100 × 750 front cover for a farming game, make sure it is pixilated like similar style to terraria or stardew valley, the title (big) "The Garden Circut" and in smaller "Keep the Greenhouse in Balance"

Prompt:

make me a prompt that asks for , first off take my existing code, make it so each square is replaced bymy plant images, but every 33% it will change into another image i have 4 images total one is when the plant is in good health, one is okay health, one is in bad health, and one image is dead, so i want it to change accordingly when health reaches those certain points, also say make the airflow more forgiving so it does better, make the grounding mechanic easier , also make it so when you water or light theres like a fraction of a second where you cant do anything because people where just spamming super fast and were easily winning, but really only a fraction of a second

Please **modify my existing p5.js Garden Circuit code** (do not rewrite from scratch). I'm going to provide the current code after this message. Implement the following changes carefully and keep everything else working (screens/states/scoring/surge/tension/etc.).

1) Replace plant-bed squares with my plant images (health-based sprites)

Right now each plant bed is drawn as a colored rectangle. Replace that visual with **images** for the plant.

I have **4 images per plant state**:

- plant_good.png (healthy)
- plant_okay.png (medium)
- plant_bad.png (low)
- plant_dead.png (wilted/dead)

Health thresholds (based on health 0–100)

- **67–100**: good image
- **34–66**: okay image
- **1–33**: bad image
- **0**: dead image

Requirements

- Use preload() to load all images.
- In each PlantBed.draw() (or wherever rendering happens), draw the correct image based on current health.
- Keep the existing UI overlays (meters/borders/selection highlight/uncertain zone shimmer/alerts) on top of the image so gameplay readability stays strong.

- If a plant is dead, it should also clearly look dead (dead image + optional desaturation overlay is fine).
- If an image fails to load, fall back gracefully to the old rectangle rendering (no crashes).

2) Make airflow more forgiving (stronger + more useful)

Players felt airflow wasn't worth using. Please buff it in at least TWO ways:

- Increase airflow **duration** (ex: +30–60% longer) AND/OR
- Increase airflow effect (ex: slows drain more, reduces health loss more, or gives a small health recovery while active)
- Optionally reduce cooldown slightly, but don't make it spammable.

Goal: airflow should feel like a meaningful stabilizer, especially during/after surges.

3) Make grounding mechanic easier (but still interactive)

Grounding rhythm is currently too strict. Make it more forgiving by:

- Widening the timing window for each beat, OR
- Reducing required perfect hits (e.g., success on 2/3 beats), OR
- Slowing the pulse slightly and making feedback clearer.

Goal: first-time players should succeed most of the time if they try, while still having a small skill element.

4) Add a tiny global "action lock" after watering or lighting (anti-spam)

Testers were spamming water/light extremely fast and winning too easily. Add a **very short input lockout** after using **Water** or **Light** actions:

- After applying water OR light, disable all actions for about **0.15–0.25 seconds** (choose a value that feels right, like 0.2s).

- During this lock, players can still move selection/cursor, but **cannot** apply water/light/airflow/grounding.
- Show subtle feedback during lock (e.g., small UI flash or dim buttons) so it feels intentional.
- Implement it cleanly as a single global timer/cooldown like actionLockTimer.

Important: this should not make the game feel laggy — it's just to stop button mashing.

Implementation notes

- Keep the code readable and commented.
- Put all new tuning values at the top as constants (e.g., ACTION_LOCK_MS, airflow buffs, grounding window).
- Do not break the title/instructions/results screens.
- After making changes, briefly summarize what files/sections you changed and which constants to tweak for balancing.

I'll paste the current code next — edit that code accordingly.

Prompt:

Q: Output format preference? A: Thanks — great questions. ### 1) Target resolution / display Please design for a **fixed desktop canvas**, not fullscreen/responsive. * **Canvas size:** **1100 × 750** (your suggestion is perfect) * Target environment is a **classroom TV / laptop browser** * Prioritize **large readable UI text** and clear spacing * **No mobile support needed** * If easy, you can center the canvas in the page and scale visually with CSS, but the internal game logic should assume a fixed canvas size. ### 2) Audience context This is for a **student mid-term class project prototype** (game design course), not a production game. So please optimize for: * **approachable, readable code** * **clear comments** * **easy tuning/editing** (constants near top) * **simple architecture** (so students can modify after playtesting) The spec is intentionally detailed

because we want something close to demo-ready, but still understandable for students. ### Extra preferences (to reduce ambiguity) * Please use **index.html + sketch.js** (preferred over a single-file HTML) * Keep visuals **simple/abstract** (no external assets) * Use **keyboard controls** (WASD/arrow + Q/E/R + Space grounding) * Build for **desktop Chrome** in a normal browser * Include the **Start Screen** and **Instructions/Explanation Screen** exactly as requested You can go ahead and build with those assumptions.

Q: Grid size for the prototype? A: 3×4 (12 beds, as spec suggests)

Q: Control scheme priority? A: Mouse only (click-based)

tml

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Garden Circuit</title>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.min.js"></script>
  <style>
    body {
      margin: 0;
      padding: 0;
      background: #1a1a2e;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      overflow: hidden;
    }
    canvas {
      display: block;
      box-shadow: 0 0 40px rgba(100, 200, 150, 0.15);
      border-radius: 4px;
    }
  </style>
</head>
<body>
  <script src="sketch.js"></script>
</body>
</html>
/ SURGE SYSTEM
// ============================================================
function startSurge() {
  surgeActive = true;
  surgeTimer = random(SURGE_DURATION_MIN, SURGE_DURATION_MAX);
  surgeVisualIntensity = 1.0;
```

```
  // Assign false alerts
  let available = beds.filter(b => !b.isWilted && b.trueUrgency !== 'critical');
  shuffle(available, true);
  let count = min(FALSE_ALERT_COUNT, available.length);
  for (let i = 0; i < count; i++) {
    available[i].hasFalseAlert = true;
  }
}

function endSurge() {
  surgeActive = false;
  surgesCompleted++;
  stats.surgesSurvived++;
  score += SCORE_SURGE_SURVIVE * comboMultiplier;

  // Clear false alerts
  for (let b of beds) b.hasFalseAlert = false;

  // Create uncertain zones
  let available = beds.filter(b => !b.isWilted);
  shuffle(available, true);
  let count = floor(random(UNCERTAIN_ZONE_COUNT_MIN, UNCERTAIN_ZONE_COUNT_MAX + 1));
  count = min(count, available.length);
  for (let i = 0; i < count; i++) {
    available[i].isUncertainZone = true;
    available[i].uncertainTimer = random(UNCERTAIN_DURATION_MIN, UNCERTAIN_DURATION_MAX);
  }

  // Set next surge timer
  let interval = lerp(SURGE_MAX_INTERVAL, SURGE_MIN_INTERVAL, (ROUND_DURATION - timer) /
ROUND_DURATION * SURGE_FREQ_RAMP);
  surgeNextIn = random(interval * 0.8, interval * 1.2);
  surgeCooldownLeft = SURGE_COOLDOWN_MIN;
}

function updateSurge(dt) {
  if (surgeActive) {
    surgeTimer -= dt;
    tensionMeter += TENSION_RISE_SURGE * dt;

    // Jitter effect
    if (!reducedEffects) {
      surgeJitterX = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
      surgeJitterY = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
    }

    if (surgeTimer <= 0) {
      endSurge();
    }
  } else {
    surgeJitterX = lerp(surgeJitterX, 0, 0.2);
    surgeJitterY = lerp(surgeJitterY, 0, 0.2);

    surgeCooldownLeft -= dt;
```

```
    surgeNextIn -= dt;

    // Tension increases surge likelihood
    let tensionBonus = tensionMeter > 60 ? 1.5 : 1.0;

    if (surgeNextIn <= 0 && surgeCooldownLeft <= 0 && timer < (ROUND_DURATION - GRACE_PERIOD)) {
      startSurge();
    }
  }

  surgeVisualIntensity = surgeActive ? lerp(surgeVisualIntensity, 1, 0.1) : lerp(surgeVisualIntensity, 0, 0.05);
}

// ==============================================================
// GROUNDING MINI-GAME
// ==============================================================
function startGrounding() {
  if (!groundingAvailable || groundingCooldownLeft > 0) return;

  prevState = gameState;
  gameState = STATE.GROUNDING;
  stats.groundingAttempts++;

  groundingBeatIndex = 0;
  groundingHits = 0;
  groundingTimer = 0;
  groundingComplete = false;
  groundingCircleRadius = 0;
  groundingFeedback = '';
  groundingFeedbackTimer = 0;

  // Generate beat timings
  groundingBeats = [];
  for (let i = 0; i < GROUNDING_BEATS; i++) {
    groundingBeats.push({
      time: 1.0 + i * GROUNDING_BEAT_INTERVAL,
      hit: false,
      result: '',
    });
  }
}

function updateGrounding(dt) {
  groundingTimer += dt;

  // Circle pulsing rhythm
  let beatPhase = (groundingTimer % GROUNDING_BEAT_INTERVAL) / GROUNDING_BEAT_INTERVAL;
  groundingCircleRadius = 30 + sin(beatPhase * PI) * 50;

  // Update feedback timer
  if (groundingFeedbackTimer > 0) groundingFeedbackTimer -= dt;

  // Check if all beats have passed
  if (groundingBeatIndex >= GROUNDING_BEATS) {
    if (!groundingComplete) {
```

```
      groundingComplete = true;
      groundingTimer = 0; // reuse as delay timer
    } else {
      groundingTimer += dt; // wait briefly
      // Actually this double-counts, let me fix
    }
  }

  if (groundingComplete) {
    // Wait a moment then return to game
    // groundingTimer was reset, so it accumulates from 0
    if (groundingTimer > 1.5) {
      finishGrounding();
    }
  } else {
    // Check if current beat window passed without input
    if (groundingBeatIndex < GROUNDING_BEATS) {
      let beat = groundingBeats[groundingBeatIndex];
      if (groundingTimer > beat.time + GROUNDING_BEAT_WINDOW + 0.1) {
        beat.result = 'missed';
        groundingFeedback = 'Missed';
        groundingFeedbackTimer = 0.6;
        groundingBeatIndex++;
      }
    }
  }

  // Still update garden decay slowly during grounding
  for (let bed of beds) {
    bed.update(dt * 0.5); // half-speed drain during grounding
  }
}

function groundingInput() {
  if (groundingComplete || groundingBeatIndex >= GROUNDING_BEATS) return;

  let beat = groundingBeats[groundingBeatIndex];
  let diff = abs(groundingTimer - beat.time);

  if (diff <= GROUNDING_BEAT_WINDOW) {
    beat.hit = true;
    beat.result = 'hit';
    groundingHits++;
    groundingFeedback = 'Steady!';
    groundingFeedbackTimer = 0.5;
  } else if (groundingTimer < beat.time - GROUNDING_BEAT_WINDOW) {
    beat.result = 'early';
    groundingFeedback = 'Too Early';
    groundingFeedbackTimer = 0.5;
  } else {
    beat.result = 'late';
    groundingFeedback = 'Too Late';
    groundingFeedbackTimer = 0.5;
  }
  groundingBeatIndex++;
```

```
}

function finishGrounding() {
  let success = groundingHits >= 2;

  if (success) {
    tensionMeter = max(0, tensionMeter - GROUNDING_TENSION_REWARD);
    groundingStabilityLeft = GROUNDING_STABILITY_TIME;
    groundingSuccessCount++;
    stats.groundingSuccesses++;
    score += SCORE_GROUNDING_BONUS * comboMultiplier;

    // Clear false alerts
    for (let b of beds) b.hasFalseAlert = false;
  } else {
    tensionMeter = max(0, tensionMeter - GROUNDING_PARTIAL_REWARD);
  }

  groundingCooldownLeft = GROUNDING_COOLDOWN;
  gameState = STATE.PLAYING;
}

// ==============================================================
// TENSION METER
// ==============================================================
function updateTension(dt) {
  // Rise from active alerts
  let activeAlerts = beds.filter(b => !b.isWilted && b.trueUrgency === 'critical').length;
  tensionMeter += activeAlerts * TENSION_RISE_ALERTS * dt * 0.3;

  // Decay during calm
  if (!surgeActive && activeAlerts < 2) {
    tensionMeter -= TENSION_DECAY * dt;
  }

  // Grounding stability calms tension faster
  if (groundingStabilityLeft > 0) {
    tensionMeter -= TENSION_DECAY * 2 * dt;
    groundingStabilityLeft -= dt;
  }

  tensionMeter = constrain(tensionMeter, 0, 100);

  // Overload at 100
  if (tensionMeter >= 100) {
    tensionMeter = TENSION_OVERLOAD_RESET;
    comboMultiplier = max(1.0, comboMultiplier - 0.5);
    // Brief visual flash handled in draw
  }
}

// ==============================================================
// COMBO SYSTEM
// ==============================================================
function updateCombo(dt) {
```

```
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40 && b.water > 30 && b.light > 30).length;

  if (healthyCount >= COMBO_THRESHOLD) {
    comboCount++;
    comboMultiplier = min(COMBO_MAX_MULTIPLIER, 1.0 + floor(comboCount / 60) *
COMBO_MULTIPLIER_STEP);
    // comboCount increments per frame roughly... let's make it time-based
  } else {
    if (comboCount > 0) {
      comboCount = max(0, comboCount - 3);
      if (comboCount === 0) comboMultiplier = 1.0;
    }
  }

  highestCombo = max(highestCombo, comboMultiplier);
  stats.peakCombo = highestCombo;
}

// ================================================================
// SCORE: per-second awards
// ================================================================
function updateScore(dt) {
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40).length;
  score += healthyCount * SCORE_PER_HEALTHY_SEC * comboMultiplier * dt;
}

// ================================================================
// DIFFICULTY RAMP
// ================================================================
function updateDifficulty() {
  let elapsed = ROUND_DURATION - timer;
  let progress = elapsed / ROUND_DURATION;
  difficultyMult = 1.0 + progress * DRAIN_RAMP_FACTOR;
}

// ================================================================
// ACTION HANDLING
// ================================================================
function doAction(type) {
  let bed = beds[selectedBed];
  if (!bed || bed.isWilted) return;

  // Tension increase for uncertain zone interaction
  if (bed.isUncertainZone) {
    tensionMeter += TENSION_RISE_UNCERTAIN;
  }

  if (type === 'water' && waterCooldown <= 0) {
    bed.applyWater();
    waterCooldown = ACTION_COOLDOWN;
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.water, 'Water');
  } else if (type === 'light' && lightCooldown <= 0) {
    bed.applyLight();
    lightCooldown = ACTION_COOLDOWN;
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.light, 'Light');
```

```
  } else if (type === 'airflow' && airflowCooldown <= 0) {
    bed.applyAirflow();
    airflowCooldown = AIRFLOW_COOLDOWN;
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.airflow, 'Airflow');
  }
}

// Process queued inputs (for surge delay)
function processInputQueue(dt) {
  for (let i = inputQueue.length - 1; i >= 0; i--) {
    inputQueue[i].delay -= dt;
    if (inputQueue[i].delay <= 0) {
      inputQueue[i].action();
      inputQueue.splice(i, 1);
    }
  }
}

function queueAction(fn) {
  if (surgeActive) {
    inputQueue.push({ action: fn, delay: SURGE_INPUT_DELAY });
  } else {
    fn();
  }
}

// ==============================================================
// CHECK WIN / LOSE
// ==============================================================
function checkEndConditions() {
  let wiltedCount = beds.filter(b => b.isWilted).length;

  if (wiltedCount >= LOSE_WILTED_COUNT) {
    gameState = STATE.LOSE;
    return;
  }

  if (timer <= 0) {
    let aliveCount = beds.filter(b => !b.isWilted).length;
    if (aliveCount >= WIN_MIN_ALIVE) {
      gameState = STATE.WIN;
    } else {
      gameState = STATE.LOSE;
    }
  }
}

// ==============================================================
// COMPUTE GRADE
// ==============================================================
function computeGrade() {
  // Based on score relative to theoretical max
  let maxPossible = TOTAL_BEDS * SCORE_PER_HEALTHY_SEC * ROUND_DURATION * 2;
  let ratio = score / maxPossible;
  if (ratio > 0.6) return 'A';
```

```
  if (ratio > 0.4) return 'B';
  if (ratio > 0.25) return 'C';
  return 'D';
}

// ==============================================================
// CLICK ON BED
// ==============================================================
function getBedAtMouse(mx, my) {
  for (let i = 0; i < beds.length; i++) {
    let b = beds[i];
    if (mx >= b.x && mx <= b.x + b.w && my >= b.y && my <= b.y + b.h) {
      return i;
    }
  }
  return -1;
}

// ==============================================================
// DRAW FUNCTIONS
// ==============================================================

// --- Title Screen ---
function drawTitle() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  // Subtle background animation
  let t = millis() / 1000;
  noStroke();
  for (let i = 0; i < 8; i++) {
    let x = CANVAS_W / 2 + cos(t * 0.3 + i * 0.8) * 200;
    let y = CANVAS_H / 2 + sin(t * 0.4 + i * 1.1) * 120;
    let r = 40 + sin(t * 0.5 + i) * 15;
    fill(COL.accent[0], COL.accent[1], COL.accent[2], 15 + sin(t + i) * 8);
    ellipse(x, y, r * 2, r * 2);
  }

  // Title
  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(52);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Garden Circuit', CANVAS_W / 2, CANVAS_H / 2 - 120);

  // Tagline
  textStyle(NORMAL);
  textSize(18);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Keep the greenhouse in balance', CANVAS_W / 2, CANVAS_H / 2 - 70);

  // Draw a decorative line
  stroke(COL.accent[0], COL.accent[1], COL.accent[2], 60);
  strokeWeight(1);
  line(CANVAS_W / 2 - 120, CANVAS_H / 2 - 45, CANVAS_W / 2 + 120, CANVAS_H / 2 - 45);
  noStroke();
```

```
}

// --- Instructions Screen ---
function drawInstructions() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let cx = CANVAS_W / 2;
  let startY = 30;

  textAlign(CENTER, TOP);
  textStyle(BOLD);
  textSize(32);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('How to Play', cx, startY);

  textStyle(NORMAL);
  textSize(15);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  textAlign(LEFT, TOP);

  let lx = 80;
  let y = startY + 55;
  let lineH = 22;

  let lines = [

  for (let line of lines) {
    if (line.startsWith('[')) {
      fill(COL.accent[0], COL.accent[1], COL.accent[2]);
      textStyle(BOLD);
      textSize(15);
    } else if (line.startsWith('  •')) {
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
      textStyle(NORMAL);
      textSize(14);
    } else {
      fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
      textStyle(NORMAL);
      textSize(14);
    }
    text(line, lx, y);
    y += lineH;
  }
}

// --- Gameplay Drawing ---
function drawGameplay() {
  // Apply surge screen shake
  if (!reducedEffects && surgeVisualIntensity > 0.1) {
    translate(surgeJitterX * surgeVisualIntensity, surgeJitterY * surgeVisualIntensity);
  }

  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  // Surge visual overlay
```

```
  if (surgeActive && !reducedEffects) {
    // Faint red/noise overlay
    noStroke();
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 15 + sin(millis() / 80) * 10);
    rect(0, 0, CANVAS_W, CANVAS_H);

    // Floating warning particles
    for (let i = 0; i < 3; i++) {
      let fx = random(boardX, boardX + boardW);
      let fy = random(boardY, boardY + boardH);
      fill(COL.surge[0], COL.surge[1], COL.surge[2], 30);
      textSize(12);
      textAlign(CENTER, CENTER);
      text('!', fx, fy);
    }
  }

  // Draw plant beds
  for (let bed of beds) {
    drawBed(bed);
  }

  // Draw selected bed highlight
  if (beds[selectedBed]) {
    let sb = beds[selectedBed];
    noFill();
    stroke(COL.accent[0], COL.accent[1], COL.accent[2], 180 + sin(millis() / 200) * 60);
    strokeWeight(3);
    rect(sb.x - 3, sb.y - 3, sb.w + 6, sb.h + 6, 6);
    noStroke();
  }

  // Draw particles
  drawParticles();

  // Draw side panel
  drawPanel();

  // Surge indicator
  if (surgeActive) {
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 180 + sin(millis() / 100) * 60);
    textAlign(CENTER, CENTER);
    textSize(20);
    textStyle(BOLD);
    text('⚡ SURGE ACTIVE ⚡', boardX + boardW / 2, boardY - 3);
    textStyle(NORMAL);
  }
}

function drawBed(bed) {
  let bx = bed.x;
  let by = bed.y;
  let bw = bed.w;
  let bh = bed.h;
```

```
// Background
if (bed.isWilted) {
  fill(COL.bedDead[0], COL.bedDead[1], COL.bedDead[2]);
} else if (bed.isUncertainZone) {
  // Subtle shimmer
  let shimmer = sin(millis() / 300 + bed.index) * 10;
  fill(COL.bedUncertain[0] + shimmer, COL.bedUncertain[1] + shimmer, COL.bedUncertain[2] + shimmer);
} else {
  fill(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]);
}

// Flash effects
if (bed.flashTimer > 0) {
  let flashAlpha = bed.flashTimer / 0.3;
  fill(lerpColor(
    color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
    color(COL.accent[0], COL.accent[1], COL.accent[2]),
    flashAlpha * 0.3
  ));
}

if (bed.restoreFlash > 0) {
  let flashAlpha = bed.restoreFlash / 0.5;
  fill(lerpColor(
    color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
    color(100, 255, 150),
    flashAlpha * 0.4
  ));
}

stroke(60, 70, 60);
strokeWeight(1);
rect(bx, by, bw, bh, 4);
noStroke();

if (bed.isWilted) {
  // Dead plant indicator
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 120);
  textAlign(CENTER, CENTER);
  textSize(28);
  text('✕', bx + bw / 2, by + bh / 2 - 10);
  textSize(12);
  text('Wilted', bx + bw / 2, by + bh / 2 + 18);
  return;
}

// Plant icon (simple)
let healthColor = bed.health > 50 ?
  COL.health :
  [lerp(COL.healthLow[0], COL.health[0], bed.health / 50),
   lerp(COL.healthLow[1], COL.health[1], bed.health / 50),
   lerp(COL.healthLow[2], COL.health[2], bed.health / 50)];

fill(healthColor[0], healthColor[1], healthColor[2]);
textAlign(CENTER, CENTER);
```

```
textSize(22);
let plantIcon = bed.health > 60 ? '❋' : bed.health > 30 ? '❊' : '✿';
text(plantIcon, bx + bw / 2, by + 24);

// Health bar
let barY = by + 44;
let barH = 7;
let barW = bw - 20;
let barX = bx + 10;

// Health bar bg
fill(30, 35, 30);
rect(barX, barY, barW, barH, 2);
// Health bar fill
let hw = barW * bed.health / 100;
fill(healthColor[0], healthColor[1], healthColor[2]);
rect(barX, barY, hw, barH, 2);

// Labels
textSize(9);
textStyle(BOLD);
fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
textAlign(CENTER, TOP);
text('HP ' + floor(bed.health), bx + bw / 2, barY + barH + 2);
textStyle(NORMAL);

// Water bar
let wBarY = barY + barH + 16;
fill(30, 35, 50);
rect(barX, wBarY, barW, barH, 2);
fill(COL.water[0], COL.water[1], COL.water[2]);
rect(barX, wBarY, barW * bed.water / 100, barH, 2);

// Water icon + value
textSize(9);
fill(COL.water[0], COL.water[1], COL.water[2]);
textAlign(LEFT, TOP);
text('💧' + floor(bed.water), barX, wBarY + barH + 2);

// Light bar
let lBarY = wBarY + barH + 16;
fill(30, 35, 30);
rect(barX, lBarY, barW, barH, 2);
fill(COL.light[0], COL.light[1], COL.light[2]);
rect(barX, lBarY, barW * bed.light / 100, barH, 2);

// Light icon + value
fill(COL.light[0], COL.light[1], COL.light[2]);
textAlign(LEFT, TOP);
text('☀' + floor(bed.light), barX, lBarY + barH + 2);

// Airflow indicator
if (bed.airflowActive) {
  fill(COL.airflow[0], COL.airflow[1], COL.airflow[2], 160);
  textAlign(RIGHT, TOP);
```

```
    text('🌬' + floor(bed.airflowTimer) + 's', barX + barW, lBarY + barH + 2);
  }

  // Status indicators
  let urgency = bed.displayUrgency;

  // Surge flicker effect on display
  if (surgeActive && !reducedEffects && random() < 0.15) {
    // Random flicker
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 50);
    noStroke();
    rect(bx, by, bw, bh, 4);
  }

  // False alert indicator
  if (bed.hasFalseAlert && !bed.isWilted) {
    fill(COL.falseAlert[0], COL.falseAlert[1], COL.falseAlert[2], 160 + sin(millis() / 150) * 60);
    textAlign(RIGHT, TOP);
    textSize(16);
    text('⚠', bx + bw - 4, by + 4);
  } else if (urgency === 'critical') {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2], 180);
    textAlign(RIGHT, TOP);
    textSize(14);
    text('!!', bx + bw - 6, by + 4);
  } else if (urgency === 'warning') {
    fill(COL.light[0], COL.light[1], COL.light[2], 140);
    textAlign(RIGHT, TOP);
    textSize(13);
    text('!', bx + bw - 6, by + 6);
  }

  // Uncertain zone indicator
  if (bed.isUncertainZone) {
    noFill();
    stroke(COL.bedUncertain[0] + 40, COL.bedUncertain[1] + 40, COL.bedUncertain[2] + 40, 100 + sin(millis() /
400 + bed.index) * 50);
    strokeWeight(2);
    rect(bx + 2, by + 2, bw - 4, bh - 4, 3);
    noStroke();

    fill(180, 160, 220, 140);
    textAlign(LEFT, BOTTOM);
    textSize(8);
    text('uncertain', bx + 5, by + bh - 3);
  }

  // Bed index (small, for debugging / identification)
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 60);
  textAlign(LEFT, TOP);
  textSize(8);
  text('#' + (bed.index + 1), bx + 4, by + 4);
}

// --- Side Panel ---
```

```
function drawPanel() {
  // Panel background
  fill(COL.panelBg[0], COL.panelBg[1], COL.panelBg[2]);
  stroke(50, 60, 50);
  strokeWeight(1);
  rect(panelX, 0, PANEL_WIDTH, CANVAS_H);
  noStroke();

  let px = panelX + 20;
  let py = 20;
  let pw = PANEL_WIDTH - 40;

  // Timer
  textAlign(LEFT, TOP);
  textStyle(BOLD);
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('TIME', px, py);
  textSize(30);
  fill(timer < 15 ? COL.healthLow[0] : COL.textPrimary[0],
       timer < 15 ? COL.healthLow[1] : COL.textPrimary[1],
       timer < 15 ? COL.healthLow[2] : COL.textPrimary[2]);
  text(floor(timer) + 's', px, py + 16);
  py += 60;

  // Score
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('SCORE', px, py);
  textSize(24);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text(floor(score), px, py + 16);
  py += 52;

  // Combo
  textSize(13);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text('HARMONY  x' + nf(comboMultiplier, 1, 1), px, py);
  // Combo bar
  py += 20;
  fill(30, 30, 20);
  rect(px, py, pw, 8, 3);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  rect(px, py, pw * ((comboMultiplier - 1) / (COMBO_MAX_MULTIPLIER - 1)), 8, 3);
  py += 24;

  // Tension Meter
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('TENSION', px, py);
  py += 18;
  fill(30, 25, 25);
  rect(px, py, pw, 14, 4);
  let tensionCol = tensionMeter < 30 ? [80, 180, 120] :
                   tensionMeter < 60 ? [200, 180, 60] :
```

```
            tensionMeter < 80 ? [220, 120, 50] :
            [200, 60, 60];
fill(tensionCol[0], tensionCol[1], tensionCol[2]);
rect(px, py, pw * tensionMeter / 100, 14, 4);
fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
textAlign(CENTER, CENTER);
textSize(10);
text(floor(tensionMeter), px + pw / 2, py + 7);
textAlign(LEFT, TOP);
py += 30;

// Surge status
textSize(13);
if (surgeActive) {
  fill(COL.surge[0], COL.surge[1], COL.surge[2]);
  textStyle(BOLD);
  text('⚡ SURGE ACTIVE', px, py);
  textStyle(NORMAL);
} else {
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('System Stable', px, py);
}
py += 28;

// Grounding status
textSize(13);
fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
text('GROUNDING', px, py);
py += 18;
if (groundingCooldownLeft > 0) {
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  textSize(12);
  text('Cooldown: ' + floor(groundingCooldownLeft) + 's', px, py);

  // Cooldown bar
  py += 16;
  fill(30, 30, 30);
  rect(px, py, pw, 8, 3);
  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2], 120);
  rect(px, py, pw * (1 - groundingCooldownLeft / GROUNDING_COOLDOWN), 8, 3);
} else {
  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
  textSize(13);
  textStyle(BOLD);
  text('Ready [SPACE]', px, py);
  textStyle(NORMAL);
  py += 16;
}
py += 24;

// Divider
stroke(60, 70, 60, 80);
strokeWeight(1);
line(px, py, px + pw, py);
noStroke();
```

```
  py += 14;

  // Selected bed info
  let sb = beds[selectedBed];
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('SELECTED BED #' + (selectedBed + 1), px, py);
  py += 20;

  if (sb && !sb.isWilted) {
    textSize(12);
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
    text('Health: ' + floor(sb.health), px, py); py += 16;
    fill(COL.water[0], COL.water[1], COL.water[2]);
    text('Water:  ' + floor(sb.water), px, py); py += 16;
    fill(COL.light[0], COL.light[1], COL.light[2]);
    text('Light:  ' + floor(sb.light), px, py); py += 16;
    if (sb.airflowActive) {
      fill(COL.airflow[0], COL.airflow[1], COL.airflow[2]);
      text('Airflow: ' + nf(sb.airflowTimer, 1, 1) + 's', px, py);
    }
    if (sb.isUncertainZone) {
      fill(180, 160, 220);
      py += 16;
      text('⬡ Uncertain Zone', px, py);
    }
  } else if (sb && sb.isWilted) {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2]);
    textSize(12);
    text('This plant has wilted.', px, py);
  }
  py += 30;

  // Action buttons (mouse clickable)
  drawActionButtons(px, py, pw);

  // Wilted count at bottom
  let wiltedCount = beds.filter(b => b.isWilted).length;
  textAlign(LEFT, BOTTOM);
  textSize(12);
  fill(wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[0] : COL.textSecondary[0],
       wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[1] : COL.textSecondary[1],
       wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[2] : COL.textSecondary[2]);
  text('Wilted: ' + wiltedCount + ' / ' + LOSE_WILTED_COUNT + ' max', px, CANVAS_H - 38);

  textSize(10);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 100);
  text('P=Pause  V=Effects', px, CANVAS_H - 18);
}

// --- Action Buttons ---
let actionBtns = [];

function drawActionButtons(px, py, pw) {
  let btnW = pw;
```

```
  let btnH = 32;
  let gap = 8;

  // Create buttons only once, then just draw them
  if (actionBtns.length === 0) {
    actionBtns.push(new Button(px, py, btnW, btnH, '💧 Water  [Q]', () => queueAction(() =>
doAction('water')), 'water'));
    actionBtns.push(new Button(px, py + btnH + gap, btnW, btnH, '☀ Light  [E]', () => queueAction(() =>
doAction('light')), 'light'));
    actionBtns.push(new Button(px, py + 2 * (btnH + gap), btnW, btnH, '🌬 Airflow  [R]', () => queueAction(()
=> doAction('airflow')), 'airflow'));
    actionBtns.push(new Button(px, py + 3 * (btnH + gap), btnW, btnH, '◉ Ground  [SPACE]', () =>
startGrounding(), 'ground'));
  }

  // Update positions each frame (in case layout changes)
  actionBtns[0].y = py;
  actionBtns[1].y = py + btnH + gap;
  actionBtns[2].y = py + 2 * (btnH + gap);
  actionBtns[3].y = py + 3 * (btnH + gap);

  // Show cooldown state
  for (let btn of actionBtns) {
    btn.checkHover(mouseX, mouseY);

    let available = true;
    if (btn.id === 'water') available = waterCooldown <= 0;
    if (btn.id === 'light') available = lightCooldown <= 0;
    if (btn.id === 'airflow') available = airflowCooldown <= 0;
    if (btn.id === 'ground') available = groundingCooldownLeft <= 0 && gameState === STATE.PLAYING;

    if (!available) {
      fill(35, 40, 38);
      stroke(50, 55, 50);
      strokeWeight(1);
      rect(btn.x, btn.y, btn.w, btn.h, 6);
      noStroke();
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 100);
      textAlign(CENTER, CENTER);
      textSize(14);
      text(btn.label, btn.x + btn.w / 2, btn.y + btn.h / 2);
    } else {
      btn.draw();
    }
  }
}

// --- Grounding Mini-Game Screen ---
function drawGrounding() {
  // Semi-transparent overlay
  fill(COL.bg[0], COL.bg[1], COL.bg[2], 200);
  rect(0, 0, CANVAS_W, CANVAS_H);

  let cx = CANVAS_W / 2;
```

```
let cy = CANVAS_H / 2;

// Title
fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
textAlign(CENTER, CENTER);
textStyle(BOLD);
textSize(24);
text('Grounding Routine', cx, cy - 140);
textStyle(NORMAL);

// Instructions
textSize(14);
fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
text('Press SPACE on each pulse beat', cx, cy - 110);

// Pulsing circle
let beatPhase;
if (groundingBeatIndex < GROUNDING_BEATS && !groundingComplete) {
  let currentBeatTime = groundingBeats[groundingBeatIndex].time;
  let distToBeat = currentBeatTime - groundingTimer;
  // Pulse grows as beat approaches
  beatPhase = 1.0 - constrain(abs(distToBeat) / (GROUNDING_BEAT_INTERVAL * 0.5), 0, 1);
} else {
  beatPhase = 0.5;
}

let radius = 30 + beatPhase * 60;

// Outer ring (target)
noFill();
stroke(COL.grounding[0], COL.grounding[1], COL.grounding[2], 80);
strokeWeight(2);
ellipse(cx, cy, 180, 180);

// Pulsing inner circle
let alpha = 120 + beatPhase * 135;
fill(COL.grounding[0], COL.grounding[1], COL.grounding[2], alpha);
stroke(COL.grounding[0], COL.grounding[1], COL.grounding[2], 200);
strokeWeight(2);
ellipse(cx, cy, radius * 2, radius * 2);
noStroke();

// Beat indicators
for (let i = 0; i < GROUNDING_BEATS; i++) {
  let bx = cx - 40 + i * 40;
  let by = cy + 120;

  if (groundingBeats[i] && groundingBeats[i].result === 'hit') {
    fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
  } else if (groundingBeats[i] && (groundingBeats[i].result === 'early' || groundingBeats[i].result === 'late' ||
groundingBeats[i].result === 'missed')) {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2], 150);
  } else if (i === groundingBeatIndex) {
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2], 180 + sin(millis() / 200) * 60);
  } else {
```

```
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 80);
    }
    ellipse(bx, by, 20, 20);
  }

  // Feedback text
  if (groundingFeedbackTimer > 0 && groundingFeedback) {
    let feedCol = groundingFeedback === 'Steady!' ? COL.grounding : COL.falseAlert;
    fill(feedCol[0], feedCol[1], feedCol[2], groundingFeedbackTimer / 0.6 * 255);
    textSize(20);
    textStyle(BOLD);
    text(groundingFeedback, cx, cy + 80);
    textStyle(NORMAL);
  }

  // Completion message
  if (groundingComplete) {
    let success = groundingHits >= 2;
    if (success) {
      fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
      textSize(22);
      textStyle(BOLD);
      text('Grounded — Clarity Restored', cx, cy - 170);
    } else {
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
      textSize(18);
      text('Partial focus — keep going', cx, cy - 170);
    }
    textStyle(NORMAL);
  }
}

// --- Win Screen ---
function drawWin() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let cx = CANVAS_W / 2;

  // Gentle celebration background
  let t = millis() / 1000;
  noStroke();
  for (let i = 0; i < 12; i++) {
    let x = cx + cos(t * 0.2 + i * 0.5) * 300;
    let y = CANVAS_H / 2 + sin(t * 0.3 + i * 0.7) * 200;
    fill(COL.accent[0], COL.accent[1], COL.accent[2], 12);
    ellipse(x, y, 60, 60);
  }

  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(42);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Greenhouse Stabilized', cx, 100);

  textStyle(NORMAL);
```

```
  textSize(18);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('You kept the garden in balance.', cx, 150);

  // Grade
  let grade = computeGrade();
  textSize(60);
  textStyle(BOLD);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text(grade, cx, 230);
  textStyle(NORMAL);

  // Stats
  drawEndStats(cx, 290);
}

// --- Lose Screen ---
function drawLose() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let cx = CANVAS_W / 2;

  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(36);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('The Garden Faded', cx, 100);

  textStyle(NORMAL);
  textSize(16);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Too many plants were lost — but every attempt is practice.', cx, 145);
  text('Adjust your strategy and try again.', cx, 168);

  drawEndStats(cx, 230);
}

function drawEndStats(cx, startY) {
  let y = startY;
  let lineH = 30;

  textSize(16);
  textAlign(CENTER, CENTER);

  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('Score: ' + floor(score), cx, y); y += lineH;

  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text('Best Harmony: x' + nf(stats.peakCombo, 1, 1), cx, y); y += lineH;

  fill(COL.surge[0], COL.surge[1], COL.surge[2]);
  text('Surges Survived: ' + stats.surgesSurvived, cx, y); y += lineH;

  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
```

```
  text('Grounding: ' + stats.groundingSuccesses + ' / ' + stats.groundingAttempts + ' successful', cx, y); y +=
lineH;

  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Plants Restored: ' + stats.totalRestores, cx, y); y += lineH;

  fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2]);
  text('Plants Wilted: ' + stats.plantsWilted, cx, y); y += lineH + 10;
}

// --- Pause Screen ---
function drawPause() {
  fill(COL.bg[0], COL.bg[1], COL.bg[2], 180);
  rect(0, 0, CANVAS_W, CANVAS_H);

  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(36);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('PAUSED', CANVAS_W / 2, CANVAS_H / 2 - 30);

  textStyle(NORMAL);
  textSize(16);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Press P to resume', CANVAS_W / 2, CANVAS_H / 2 + 20);
}

// ==============================================================
// P5.JS SETUP
// ==============================================================
function setup() {
  createCanvas(CANVAS_W, CANVAS_H);
  textFont('monospace');
  initGame();
  initMenuButtons();
}

// ==============================================================
// MENU BUTTON SETUP
// ==============================================================
let menuButtons = [];

function initMenuButtons() {
  menuButtons = [];

  let cx = CANVAS_W / 2;
  let btnW = 200;
  let btnH = 48;

  // Title screen buttons
  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H / 2 - 10, btnW, btnH, 'Start Game', () => {
    initGame();
    gameState = STATE.PLAYING;
  }, 'title_start'));
```

```
  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H / 2 + 55, btnW, btnH, 'Instructions', () => {
    gameState = STATE.INSTRUCTIONS;
  }, 'title_instructions'));

  // Instructions back button
  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 60, btnW, btnH, 'Back to Title', () => {
    gameState = STATE.TITLE;
  }, 'instr_back'));

  // Win/Lose restart buttons
  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 120, btnW, btnH, 'Play Again', () => {
    actionBtns = []; // Reset action buttons
    initGame();
    gameState = STATE.PLAYING;
  }, 'end_restart'));

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 60, btnW, btnH, 'Title Screen', () => {
    actionBtns = [];
    gameState = STATE.TITLE;
  }, 'end_title'));
}

// ==============================================================
// P5.JS DRAW LOOP
// ==============================================================
function draw() {
  let dt = deltaTime / 1000;
  dt = min(dt, 0.05); // Cap delta time to avoid jumps

  // Update hover on menu buttons
  for (let btn of menuButtons) {
    btn.visible = false;
    btn.checkHover(mouseX, mouseY);
  }

  switch (gameState) {
    case STATE.TITLE:
      drawTitle();
      // Show title buttons
      showButton('title_start');
      showButton('title_instructions');
      break;

    case STATE.INSTRUCTIONS:
      drawInstructions();
      showButton('instr_back');
      break;

    case STATE.PLAYING:
      updateGame(dt);
      drawGameplay();
      break;

    case STATE.GROUNDING:
      // Still draw gameplay underneath
```

```
      drawGameplay();
      updateGrounding(dt);
      drawGrounding();
      break;

    case STATE.WIN:
      drawWin();
      showButton('end_restart');
      showButton('end_title');
      break;

    case STATE.LOSE:
      drawLose();
      showButton('end_restart');
      showButton('end_title');
      break;

    case STATE.PAUSED:
      drawGameplay();
      drawPause();
      break;
  }

  // Draw visible menu buttons
  for (let btn of menuButtons) {
    if (btn.visible) btn.draw();
  }
}

function showButton(id) {
  for (let btn of menuButtons) {
    if (btn.id === id) {
      btn.visible = true;
      btn.checkHover(mouseX, mouseY);
    }
  }
}

// ==============================================================
// MAIN UPDATE LOOP
// ==============================================================
function updateGame(dt) {
  // Timer
  timer -= dt;
  timer = max(0, timer);

  // Difficulty ramp
  updateDifficulty();

  // Update cooldowns
  if (waterCooldown > 0) waterCooldown -= dt;
  if (lightCooldown > 0) lightCooldown -= dt;
  if (airflowCooldown > 0) airflowCooldown -= dt;
  if (groundingCooldownLeft > 0) groundingCooldownLeft -= dt;
```

```
  // Process input queue (surge delay)
  processInputQueue(dt);

  // Update all plant beds
  for (let bed of beds) {
    bed.update(dt);
  }

  // Update surge
  updateSurge(dt);

  // Update tension
  updateTension(dt);

  // Update combo
  updateCombo(dt);

  // Update score
  updateScore(dt);

  // Update particles
  updateParticles(dt);

  // Check end conditions
  checkEndConditions();
}

// ============================================================
// INPUT HANDLING
// ============================================================
function keyPressed() {
  // Global keys
  if (key === 'v' || key === 'V') {
    reducedEffects = !reducedEffects;
    return;
  }

  switch (gameState) {
    case STATE.TITLE:
      if (keyCode === ENTER || key === ' ') {
        initGame();
        gameState = STATE.PLAYING;
      }
      if (key === 'i' || key === 'I') {
        gameState = STATE.INSTRUCTIONS;
      }
      break;

    case STATE.INSTRUCTIONS:
      if (keyCode === ESCAPE || keyCode === BACKSPACE || key === 'i' || key === 'I') {
        gameState = STATE.TITLE;
      }
      break;

    case STATE.PLAYING:
```

```
      handlePlayingInput();
      break;

    case STATE.GROUNDING:
      if (key === ' ') {
        groundingInput();
      }
      break;

    case STATE.WIN:
    case STATE.LOSE:
      if (keyCode === ENTER || key === ' ') {
        actionBtns = [];
        initGame();
        gameState = STATE.PLAYING;
      }
      if (keyCode === ESCAPE) {
        actionBtns = [];
        gameState = STATE.TITLE;
      }
      break;

    case STATE.PAUSED:
      if (key === 'p' || key === 'P') {
        gameState = STATE.PLAYING;
      }
      break;
  }
}

function handlePlayingInput() {
  // Movement — WASD or arrow keys
  let col = selectedBed % GRID_COLS;
  let row = floor(selectedBed / GRID_COLS);

  if (key === 'w' || key === 'W' || keyCode === UP_ARROW) {
    row = max(0, row - 1);
  }
  if (key === 's' || key === 'S' || keyCode === DOWN_ARROW) {
    row = min(GRID_ROWS - 1, row + 1);
  }
  if (key === 'a' || key === 'A' || keyCode === LEFT_ARROW) {
    col = max(0, col - 1);
  }
  if (key === 'd' || key === 'D' || keyCode === RIGHT_ARROW) {
    col = min(GRID_COLS - 1, col + 1);
  }
  selectedBed = row * GRID_COLS + col;

  // Actions
  if (key === 'q' || key === 'Q') {
    queueAction(() => doAction('water'));
  }
  if (key === 'e' || key === 'E') {
    queueAction(() => doAction('light'));
```

```
  }
  if (key === 'r' || key === 'R') {
    queueAction(() => doAction('airflow'));
  }
  if (key === ' ') {
    startGrounding();
  }

  // Pause
  if (key === 'p' || key === 'P') {
    gameState = STATE.PAUSED;
  }
}

function mousePressed() {
  // Check menu buttons
  for (let btn of menuButtons) {
    if (btn.checkClick(mouseX, mouseY)) return;
  }

  // Check action buttons during gameplay
  if (gameState === STATE.PLAYING) {
    for (let btn of actionBtns) {
      if (btn.checkClick(mouseX, mouseY)) return;
    }

    // Click on bed to select
    let idx = getBedAtMouse(mouseX, mouseY);
    if (idx >= 0) {
      selectedBed = idx;
    }
  }
}

// Prevent default scrolling for arrow keys and space
function keyDown(e) {
  if ([32, 37, 38, 39, 40].includes(e.keyCode)) {
    e.preventDefault();
  }
}

// Attach keydown listener for preventing scroll
if (typeof window !== 'undefined') {
  window.addEventListener('keydown', function(e) {
    if ([32, 37, 38, 39, 40].includes(e.keyCode)) {
      e.preventDefault();
    }
  });
}
```

How to Run

1. Save both files (`index.html` and `sketch.js`) in the same folder.

2. Open `index.html` in a desktop browser (Chrome recommended). You can use a simple local server (`python -m http.server` or VS Code Live Server) or just open the file directly — p5.js loads from CDN.
3. The game starts on the **Title Screen**. Click **Instructions** for controls and design notes, then **Start Game** to play.

Controls

Key	Action
WASD / Arrow Keys	Move selection cursor
Click a bed	Select it directly
Q / Water button	Route water to selected bed
E / Light button	Route light to selected bed
R / Airflow button	Apply temporary airflow stabilizer
Space / Ground button	Begin Grounding Routine
P	Pause / Resume
V	Toggle reduced visual effects

Where to Tweak Balance

All tuning constants are at the **top of `sketch.js`** (lines ~10–90), clearly labeled with comments. Key ones to try first:

- `ROUND_DURATION` — game length (default 90s)
- `BASE_WATER_DRAIN` / `BASE_LIGHT_DRAIN` — how fast resources deplete
- `WATER_BOOST` / `LIGHT_BOOST` — how much each action helps
- `SURGE_MIN_INTERVAL` / `SURGE_MAX_INTERVAL` — surge frequency
- `GROUNDING_COOLDOWN` / `GROUNDING_TENSION_REWARD` — grounding balance
- `LOSE_WILTED_COUNT` — how many plant deaths trigger a loss
- `DRAIN_RAMP_FACTOR` — how much harder it gets over time

The game is roughly **~650 lines** of well-commented code, organized with classes and clear state management so students can read, modify, and extend it after playtesting.

Great progress — a couple of important fixes/tuning changes:

1) Instructions screen layout bug

On the **Instructions / How to Play** screen, the **“Back to Title”** button is overlapping the bottom text (the grounding section bullets). Please **move the button so it does not cover any text**.

Preferred fix:

- Place the button **below all instruction text** with clear spacing, OR
- Move it to the **bottom-right corner** with enough margin, and ensure the text area stops above it.

  Please make sure **all instruction text is fully visible and readable** on the 1100x750 canvas.

  ### 2) Gameplay difficulty tuning (too hard right now)

  Please make the game **easier to manage** by slowing depletion rates.

  Specifically:

- Make **Water and Light deplete slower** for all beds (reduce base drain rates noticeably)
- Keep the game challenging, but less overwhelming in the first 30–45 seconds
- If needed, slightly reduce:
  - number of simultaneous critical alerts
  - how fast multiple beds spiral at once

  Goal: it should feel tense but manageable for first-time players in a class demo.

  ### Suggested tuning direction (if helpful)

- Reduce base water/light drain by ~25–40%
- Slightly slower difficulty ramp over time
- Keep the same mechanics and structure, just improve playability/balance

  Please update the code with these changes. (just place the sections that are needed for changing)

Show less
Mar 4

Here are the targeted changes:

Fix 1: Tuning constants (easier difficulty)

Replace the drain/difficulty constants near the top of `sketch.js`:

js
```
// FIND AND REPLACE THESE VALUES:

// Plant drain rates — REDUCED for easier gameplay
const BASE_WATER_DRAIN = 3.2;      // was 5.0
const BASE_LIGHT_DRAIN = 2.8;      // was 4.5
const DRAIN_VARIATION = 0.8;       // was 1.5 — less random spread
```

```
const HEALTH_LOSS_RATE = 5.5;      // was 8.0 — slower health decay
const HEALTH_RECOVERY_RATE = 3.0;   // was 2.0 — faster recovery

// Resource actions — slightly more generous
const WATER_BOOST = 35;            // was 30
const LIGHT_BOOST = 35;            // was 30

// Difficulty ramp — gentler curve
const DRAIN_RAMP_FACTOR = 0.35;    // was 0.6
const SURGE_FREQ_RAMP = 0.25;      // was 0.4

// Surge timing — more breathing room
const SURGE_MIN_INTERVAL = 16;     // was 12
const SURGE_MAX_INTERVAL = 30;     // was 25
const GRACE_PERIOD = 20;           // was 15

// Tension — slower buildup
const TENSION_RISE_SURGE = 8;      // was 12
const TENSION_RISE_UNCERTAIN = 4;  // was 6
const TENSION_RISE_ALERTS = 1.2;   // was 2
```

Fix 2: Instructions screen layout

Replace the entire `drawInstructions()` function:

js

```
function drawInstructions() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let cx = CANVAS_W / 2;
  let startY = 22;
  // Reserve space for button at bottom (button is 48px tall + 16px margin)
  let maxTextY = CANVAS_H - 80;

  textAlign(CENTER, TOP);
  textStyle(BOLD);
  textSize(28);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('How to Play', cx, startY);

  textStyle(NORMAL);
  textSize(13);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  textAlign(LEFT, TOP);

  // Two-column layout to fit everything
  let colW = 480;
  let leftX = 40;
  let rightX = CANVAS_W / 2 + 30;
  let lineH = 18;

  // ---- LEFT COLUMN ----
  let y = startY + 45;
```

```
let leftLines = [
  { t: '[ OBJECTIVE ]', style: 'header' },
  { t: 'Keep 12 plant beds healthy for 90 seconds.', style: 'normal' },
  { t: 'Plants need Water and Light — both drain over time.', style: 'normal' },
  { t: 'If needs drop too low, health decreases. Lose 4 plants = game over.', style: 'normal' },
  { t: '', style: 'normal' },
  { t: '[ CONTROLS ]', style: 'header' },
  { t: 'Arrow Keys / WASD — Move selection cursor', style: 'normal' },
  { t: 'Click a bed — Select it directly', style: 'normal' },
  { t: 'Q  or  Water btn — Route water to selected bed', style: 'normal' },
  { t: 'E  or  Light btn — Route light to selected bed', style: 'normal' },
  { t: 'R  or  Airflow btn — Temporary stabilizer (cooldown)', style: 'normal' },
  { t: 'Space — Begin Grounding Routine (when ready)', style: 'normal' },
  { t: 'P — Pause / Resume     V — Toggle reduced effects', style: 'sub' },
  { t: '', style: 'normal' },
  { t: '[ SCORING & HARMONY ]', style: 'header' },
  { t: 'Earn points by keeping beds healthy over time.', style: 'normal' },
  { t: 'Restore critical plants for bonus points.', style: 'normal' },
  { t: 'Keep 8+ beds healthy to build a Harmony combo.', style: 'normal' },
  { t: 'Higher combos multiply your score!', style: 'normal' },
];

for (let line of leftLines) {
  if (line.style === 'header') {
    fill(COL.accent[0], COL.accent[1], COL.accent[2]);
    textStyle(BOLD);
    textSize(13);
  } else if (line.style === 'sub') {
    fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
    textStyle(NORMAL);
    textSize(12);
  } else {
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
    textStyle(NORMAL);
    textSize(13);
  }
  text(line.t, leftX, y);
  y += lineH;
}

// ---- RIGHT COLUMN ----
y = startY + 45;

let rightLines = [
  { t: '[ SURGES — System Disruptions ]', style: 'header' },
  { t: 'At unpredictable intervals, a Surge disrupts the', style: 'normal' },
  { t: 'greenhouse for a few seconds:', style: 'normal' },
  { t: '  · Some indicators flicker with false urgency alerts', style: 'sub' },
  { t: '  · Visual clutter increases briefly', style: 'sub' },
  { t: '  · Controls have a slight delay', style: 'sub' },
  { t: 'Stay focused and adapt — surges are manageable.', style: 'normal' },
  { t: '', style: 'normal' },
  { t: '[ UNCERTAIN ZONES — Post-Surge ]', style: 'header' },
  { t: 'After a surge, some beds become Uncertain Zones:', style: 'normal' },
```

```
  { t: '  · Interacting with them raises Tension faster', style: 'sub' },
  { t: '  · Avoiding them may leave plants under-served', style: 'sub' },
  { t: 'Balance efficiency against rising tension.', style: 'normal' },
  { t: '', style: 'normal' },
  { t: '[ GROUNDING — Coping & Recovery ]', style: 'header' },
  { t: 'Press Space to start a Grounding Routine:', style: 'normal' },
  { t: '  · Match 3 slow pulse beats by pressing Space in rhythm', style: 'sub' },
  { t: '  · Success: reduces Tension, clears false alerts,', style: 'sub' },
  { t: '    and stabilizes the garden briefly', style: 'sub' },
  { t: '  · Has a cooldown — use it strategically', style: 'sub' },
  { t: '', style: 'normal' },
  { t: '[ A NOTE ON DESIGN ]', style: 'header' },
  { t: 'This game uses symbolic mechanics — surges, uncertainty,', style: 'normal' },
  { t: 'and grounding — to respectfully represent themes of', style: 'normal' },
  { t: 'disruption and coping. It is not a literal simulation.', style: 'normal' },
];

for (let line of rightLines) {
  if (line.style === 'header') {
    fill(COL.accent[0], COL.accent[1], COL.accent[2]);
    textStyle(BOLD);
    textSize(13);
  } else if (line.style === 'sub') {
    fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
    textStyle(NORMAL);
    textSize(12);
  } else {
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
    textStyle(NORMAL);
    textSize(13);
  }
  text(line.t, rightX, y);
  y += lineH;
}

// Divider line between columns
stroke(COL.accent[0], COL.accent[1], COL.accent[2], 40);
strokeWeight(1);
line(CANVAS_W / 2 + 10, startY + 50, CANVAS_W / 2 + 10, maxTextY - 10);
noStroke();
}
```

Then update the button position in `initMenuButtons()` — find the instructions back button and change its Y:

js

```
// Instructions back button — centered at bottom with clear margin
menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 55, btnW, btnH, 'Back to Title', () => {
  gameState = STATE.TITLE;
}, 'instr_back'));
```

That's `CANVAS_H - 55` instead of `CANVAS_H - 60`, but the real fix is the two-column layout above — all text now ends well above the button. The button sits in clear space at the bottom.

Summary of changes

Difficulty: drain rates cut ~35%, recovery boosted 50%, ramp is gentler, surges come less often with a longer grace period. First-time players should be able to keep up comfortably for the first 30–45 seconds before things get interesting.

Instructions: switched to a two-column layout so everything fits on the 1100×750 canvas with the button clearly below all content — no overlap.

does grounding pause the game, because i dont want you to lose plants while grounding, also remind me what does greounding do

Mar 4

Good catch — right now grounding **doesn't fully pause** plant decay. It slows drain to half speed, so you can still lose plants mid-routine. If you want it fully paused, that's a one-line fix.

In `updateGrounding(dt)`, find:

js
```
// Still update garden decay slowly during grounding
for (let bed of beds) {
  bed.update(dt * 0.5); // half-speed drain during grounding
}
```

Replace with:

js
```
// Pause plant decay entirely during grounding
// (plants are frozen so the player isn't punished for coping)
```

That removes the bed update call entirely, so nothing drains while you're in the rhythm mini-game.

What grounding does — quick recap:

Trigger: Press **Space** (or click the Ground button) when it's off cooldown.

What happens: A centered pulsing circle appears. You press **Space** three times, trying to hit each slow beat in rhythm (timing window). You get feedback — "Steady!", "Too Early", "Too Late".

If you hit 2 or 3 of 3 beats (success):

- Tension meter drops by 30

- All false alerts are cleared
- Garden indicators stabilize for ~5 seconds (no false alerts, calmer UI)
- You get a score bonus

If you miss (partial/fail):

- Small tension reduction (10) or none
- No stability bonus

Tradeoff: It has a **10-second cooldown** after use, and while you're doing it, you can't water/light/airflow any beds. So it's a strategic pause — you're choosing to invest a few seconds into recovery instead of managing plants.

The design intent is that grounding represents **coping as an active, deliberate choice** with real benefit, not a free reset.

can you make the game a 3x3

Mar 4

Sure — here are the changes needed:

Grid and related thresholds

Find these constants near the top of `sketch.js` and replace:

js
```
// Grid layout
const GRID_COLS = 3;            // was 4
const GRID_ROWS = 3;
const TOTAL_BEDS = GRID_COLS * GRID_ROWS;  // now 9
```
js
```
// Win/Lose — adjusted for 9 beds
const LOSE_WILTED_COUNT = 3;       // was 4
const WIN_MIN_ALIVE = 7;          // was 9 (TOTAL_BEDS - 3)
```
js
```
// Combo — easier to maintain with fewer beds
const COMBO_THRESHOLD = 6;        // was 8
```

Instructions text

In `drawInstructions()`, update the objective line in `leftLines`:

js
```
  { t: 'Keep 9 plant beds healthy for 90 seconds.', style: 'normal' },
```

And the combo line:

js

```
{ t: 'Keep 6+ beds healthy to build a Harmony combo.', style: 'normal' },
```

That's it — everything else (layout calculation, bed sizing, selection logic) is already driven by `GRID_COLS` and `GRID_ROWS`, so the board will automatically resize to a 3×3 grid with larger beds.

also how does one win

Mar 4

You win by **surviving the full 90 seconds** without losing too many plants.

Specifically, when the timer hits 0:

- If **7 or more** of your 9 plants are still alive → **Win screen** (with score, grade, stats)
- If fewer than 7 are alive → **Lose screen**

You can also **lose early** if 3 plants wilt before the timer runs out.

So the goal is: keep at least 7 of 9 beds from dying for 90 seconds, while managing surges, uncertain zones, and tension. Higher score and harmony combos earn you a better letter grade (A/B/C/D) on the win screen, but any survival with enough plants alive counts as a win.

Want me to adjust any of those thresholds — like requiring all 9 alive, or making the round shorter/longer?

Files hidden in shared chats

Please modify my existing p5.js Garden Circuit code (do not rewrite from scratch). I'm going to provide the current code after this message. Implement the following changes carefully and keep everything else working (screens/states/scoring/surge/tension/etc.). Assets note: I will place all image files inside `assets/` (a folder containing all game assets). Please load images from that folder (e.g., `assets/plant_good.png`). 1) Replace plant-bed squares with my plant images (health-based sprites) Right now each plant bed is drawn as a colored rectangle. Replace that visual with images for the plant. I have 4 images per plant state: * `plant_good.png` (healthy) * `plant_okay.png` (medium) * `plant_bad.png` (low) * `plant_dead.png` (wilted/dead) Health thresholds (based on health 0–100) * 67–100: good image * 34–66: okay image * 1–33: bad image * 0: dead image Requirements * Use `preload()` to load all images from `assets/`. * In each `PlantBed.draw()` (or wherever rendering happens), draw the correct image based on current `health`. * Keep the existing UI overlays (meters/borders/selection highlight/uncertain zone shimmer/alerts) on top of the image so gameplay readability stays strong. * If a plant is dead, it should also clearly look dead (dead image + optional desaturation overlay is fine). * If an image fails to load, fall back gracefully to the old rectangle rendering (no crashes). 2) Make airflow more forgiving (stronger + more useful) Players felt airflow wasn't worth using. Please buff it in at least TWO ways: * Increase airflow duration (ex: +30–60% longer) AND/OR * Increase airflow effect (ex: slows drain more, reduces health loss more, or gives a small health

recovery while active) * Optionally reduce cooldown slightly, but don't make it spammable. Goal: airflow should feel like a meaningful stabilizer, especially during/after surges. 3) Make grounding mechanic easier (but still interactive) Grounding rhythm is currently too strict. Make it more forgiving by: * Widening the timing window for each beat, OR * Reducing required perfect hits (e.g., success on 2/3 beats), OR * Slowing the pulse slightly and making feedback clearer. Goal: first-time players should succeed most of the time if they try, while still having a small skill element. 4) Add a tiny global "action lock" after watering or lighting (anti-spam) Testers were spamming water/light extremely fast and winning too easily. Add a very short input lockout after using Water or Light actions: * After applying water OR light, disable all actions for about 0.15–0.25 seconds (choose a value that feels right, like 0.2s). * During this lock, players can still move selection/cursor, but cannot apply water/light/airflow/grounding. * Show subtle feedback during lock (e.g., small UI flash or dim buttons) so it feels intentional. * Implement it cleanly as a single global timer/cooldown like `actionLockTimer`. Important: this should not make the game feel laggy — it's just to stop button mashing. Implementation notes * Keep the code readable and commented. * Put all new tuning values at the top as constants (e.g., `ACTION_LOCK_MS`, airflow buffs, grounding window). * Do not break the title/instructions/results screens. * After making changes, briefly summarize what files/sections you changed and which constants to tweak for balancing. I'll paste the current code next, edit that code accordingly.

Show less
Mar 11

Here's the fully modified `sketch.js` with all four changes integrated. I'll call out each change with `// CHANGE #N` comments throughout.

js
```
// ============================================================
// GARDEN CIRCUIT — p5.js Prototype
// A real-time greenhouse management game with symbolic
// mechanics representing disruption, uncertainty, and coping.
// ============================================================

// ============================================================
// TUNING CONSTANTS — edit these to balance difficulty
// ============================================================

const CANVAS_W = 1100;
const CANVAS_H = 750;

// Grid layout
const GRID_COLS = 3;
const GRID_ROWS = 3;
const TOTAL_BEDS = GRID_COLS * GRID_ROWS;

// Round / timing
const ROUND_DURATION = 90;
const GRACE_PERIOD = 15;

// Plant drain rates
const BASE_WATER_DRAIN = 5;
```

```
const BASE_LIGHT_DRAIN = 4;
const DRAIN_VARIATION = 1.5;
const HEALTH_LOSS_RATE = 7;
const HEALTH_RECOVERY_RATE = 2.5;
const CRITICAL_THRESHOLD = 25;
const HEALTHY_THRESHOLD = 50;

// Resource actions
const WATER_BOOST = 30;
const LIGHT_BOOST = 30;

// CHANGE #2 — Buffed airflow constants
const AIRFLOW_DURATION = 8;            // was 5 — lasts 60% longer
const AIRFLOW_DRAIN_REDUCTION = 0.3;    // was 0.5 — slows drain to 30% (stronger)
const AIRFLOW_HEALTH_RECOVERY = 1.5;    // NEW — heals 1.5 HP/sec while airflow active
const AIRFLOW_COOLDOWN = 3;            // was 4 — slightly shorter cooldown

const ACTION_COOLDOWN = 0.3;

// CHANGE #4 — Global action lock (anti-spam)
const ACTION_LOCK_DURATION = 0.2;       // NEW — 0.2s lockout after water/light

// Surge system
const SURGE_MIN_INTERVAL = 12;
const SURGE_MAX_INTERVAL = 25;
const SURGE_DURATION_MIN = 4;
const SURGE_DURATION_MAX = 7;
const SURGE_COOLDOWN_MIN = 8;
const FALSE_ALERT_COUNT = 3;
const SURGE_INPUT_DELAY = 0.1;
const SURGE_JITTER_AMOUNT = 3;

// Uncertain zones
const UNCERTAIN_ZONE_COUNT_MIN = 2;
const UNCERTAIN_ZONE_COUNT_MAX = 4;
const UNCERTAIN_DURATION_MIN = 8;
const UNCERTAIN_DURATION_MAX = 15;

// Tension meter
const TENSION_RISE_SURGE = 12;
const TENSION_RISE_UNCERTAIN = 4;
const TENSION_RISE_ALERTS = 1.2;
const TENSION_DECAY = 3;
const TENSION_OVERLOAD_RESET = 70;

// CHANGE #3 — More forgiving grounding constants
const GROUNDING_COOLDOWN = 10;
const GROUNDING_BEATS = 3;
const GROUNDING_BEAT_WINDOW = 0.45;     // was 0.3 — 50% wider timing window
const GROUNDING_BEAT_INTERVAL = 1.5;    // was 1.2 — slower rhythm, easier to read
const GROUNDING_SUCCESS_THRESHOLD = 1;  // NEW — only need 1/3 hits to succeed (was hardcoded 2)
const GROUNDING_TENSION_REWARD = 30;
const GROUNDING_STABILITY_TIME = 5;
const GROUNDING_PARTIAL_REWARD = 10;
```

```
// CHANGE #1 — Plant image health thresholds
const PLANT_IMG_GOOD_MIN = 67;        // NEW — health >= 67 shows good image
const PLANT_IMG_OKAY_MIN = 34;        // NEW — health >= 34 shows okay image
const PLANT_IMG_BAD_MIN = 1;          // NEW — health >= 1 shows bad image
                                      // health <= 0 shows dead image

// Scoring
const SCORE_PER_HEALTHY_SEC = 1;
const SCORE_RESTORE_BONUS = 15;
const SCORE_GROUNDING_BONUS = 25;
const SCORE_SURGE_SURVIVE = 20;
const COMBO_THRESHOLD = 6;
const COMBO_MULTIPLIER_STEP = 0.1;
const COMBO_MAX_MULTIPLIER = 3.0;

// Win/Lose
const LOSE_WILTED_COUNT = 3;
const WIN_MIN_ALIVE = 7;

// Difficulty ramp
const DRAIN_RAMP_FACTOR = 0.5;
const SURGE_FREQ_RAMP = 0.4;

// Visual
const BED_MARGIN = 12;
const PANEL_WIDTH = 260;

// ============================================================
// COLOR PALETTE
// ============================================================
const COL = {
  bg:          [26, 30, 46],
  panelBg:     [20, 24, 38],
  bedNormal:   [40, 55, 45],
  bedSelected: [60, 90, 70],
  bedUncertain:[55, 50, 65],
  bedDead:     [50, 35, 35],
  health:      [80, 200, 120],
  healthLow:   [200, 80, 80],
  water:       [80, 150, 220],
  light:       [230, 200, 80],
  airflow:     [180, 210, 230],
  tension:     [200, 100, 100],
  textPrimary: [230, 235, 240],
  textSecondary:[160, 170, 180],
  surge:       [180, 60, 60],
  grounding:   [100, 180, 160],
  combo:       [240, 200, 80],
  accent:      [100, 200, 150],
  falseAlert:  [220, 130, 60],
  buttonBg:    [50, 70, 60],
  buttonHover: [70, 100, 80],
  buttonText:  [230, 240, 230],
};
```

```
// ============================================================
// GAME STATE
// ============================================================
const STATE = {
  TITLE: 'title',
  INSTRUCTIONS: 'instructions',
  PLAYING: 'playing',
  GROUNDING: 'grounding',
  WIN: 'win',
  LOSE: 'lose',
  PAUSED: 'paused',
};

let gameState = STATE.TITLE;
let prevState = STATE.TITLE;

// Game variables
let beds = [];
let selectedBed = 0;
let timer = ROUND_DURATION;
let score = 0;
let tensionMeter = 0;
let comboCount = 0;
let comboMultiplier = 1.0;
let highestCombo = 0;

// Surge state
let surgeActive = false;
let surgeTimer = 0;
let surgeNextIn = 0;
let surgeCooldownLeft = 0;
let surgesCompleted = 0;
let surgeVisualIntensity = 0;

// Grounding state
let groundingAvailable = true;
let groundingCooldownLeft = 0;
let groundingSuccessCount = 0;
let groundingStabilityLeft = 0;

// Grounding mini-game state
let groundingBeats = [];
let groundingBeatIndex = 0;
let groundingPhase = 0;
let groundingCircleRadius = 0;
let groundingHits = 0;
let groundingTimer = 0;
let groundingFeedback = '';
let groundingFeedbackTimer = 0;
let groundingComplete = false;

// Action cooldowns
let waterCooldown = 0;
let lightCooldown = 0;
let airflowCooldown = 0;
```

```
// CHANGE #4 — Global action lock timer
let actionLockTimer = 0;

// Input delay queue (for surge effect)
let inputQueue = [];
let surgeJitterX = 0;
let surgeJitterY = 0;

// Particles
let particles = [];
const MAX_PARTICLES = 60;

// Stats for end screen
let stats = {
  surgesSurvived: 0,
  groundingAttempts: 0,
  groundingSuccesses: 0,
  plantsWilted: 0,
  peakCombo: 0,
  totalRestores: 0,
};

let titlePulses = [];
let buttons = [];
let difficultyMult = 1.0;
let reducedEffects = false;

// ==============================================================
// CHANGE #1 — Plant image variables
// ==============================================================
let plantGoodImg = null;
let plantOkayImg = null;
let plantBadImg = null;
let plantDeadImg = null;
let imagesLoaded = false;  // fallback flag if images fail

// ==============================================================
// CHANGE #1 — Preload plant images from assets/ folder
// ==============================================================
function preload() {
  // Attempt to load all 4 plant state images.
  // If any fail, imagesLoaded stays false and we fall back
  // to the old rectangle rendering (no crash).
  try {
    plantGoodImg = loadImage('assets/plant_good.png',
      () => {}, // success callback (no-op)
      () => { plantGoodImg = null; } // failure callback
    );
    plantOkayImg = loadImage('assets/plant_okay.png',
      () => {},
      () => { plantOkayImg = null; }
    );
    plantBadImg = loadImage('assets/plant_bad.png',
      () => {},
```

```
      () => { plantBadImg = null; }
    );
    plantDeadImg = loadImage('assets/plant_dead.png',
      () => {},
      () => { plantDeadImg = null; }
    );
  } catch (e) {
    console.warn('Plant images could not be loaded, using fallback rendering.', e);
  }
}

// Helper: check after setup whether all images loaded successfully
function checkImagesLoaded() {
  imagesLoaded = (plantGoodImg !== null && plantOkayImg !== null &&
                  plantBadImg !== null && plantDeadImg !== null);
  if (!imagesLoaded) {
    console.warn('One or more plant images missing — using fallback shapes.');
  }
}

// Helper: get the correct image for a given health value
function getPlantImage(health) {
  if (health <= 0) return plantDeadImg;
  if (health < PLANT_IMG_OKAY_MIN) return plantBadImg;
  if (health < PLANT_IMG_GOOD_MIN) return plantOkayImg;
  return plantGoodImg;
}

// ==============================================================
// PLANT BED CLASS
// ==============================================================
class PlantBed {
  constructor(col, row, index) {
    this.col = col;
    this.row = row;
    this.index = index;

    this.health = 80 + random(-10, 10);
    this.water = 60 + random(-15, 15);
    this.light = 60 + random(-15, 15);

    this.drainRateWater = BASE_WATER_DRAIN + random(-DRAIN_VARIATION, DRAIN_VARIATION);
    this.drainRateLight = BASE_LIGHT_DRAIN + random(-DRAIN_VARIATION, DRAIN_VARIATION);

    this.airflowActive = false;
    this.airflowTimer = 0;

    this.isUncertainZone = false;
    this.uncertainTimer = 0;

    this.hasFalseAlert = false;
    this.isWilted = false;

    this.x = 0;
    this.y = 0;
```

```
  this.w = 0;
  this.h = 0;

  this.flashTimer = 0;
  this.restoreFlash = 0;
  this.wasStressed = false;
}

get trueUrgency() {
  if (this.health <= 0) return 'dead';
  if (this.water < CRITICAL_THRESHOLD || this.light < CRITICAL_THRESHOLD) return 'critical';
  if (this.water < HEALTHY_THRESHOLD || this.light < HEALTHY_THRESHOLD) return 'warning';
  return 'healthy';
}

get displayUrgency() {
  if (this.hasFalseAlert) return 'critical';
  if (this.isUncertainZone && surgeActive) {
    if (random() < 0.2) {
      return random() < 0.5 ? 'warning' : 'healthy';
    }
  }
  return this.trueUrgency;
}

update(dt) {
  if (this.isWilted) return;

  let drainMult = difficultyMult;
  if (this.airflowActive) drainMult *= AIRFLOW_DRAIN_REDUCTION;

  this.water -= this.drainRateWater * drainMult * dt;
  this.light -= this.drainRateLight * drainMult * dt;
  this.water = constrain(this.water, 0, 100);
  this.light = constrain(this.light, 0, 100);

  if (this.water < CRITICAL_THRESHOLD || this.light < CRITICAL_THRESHOLD) {
    // CHANGE #2 — Airflow reduces health loss when plant is stressed
    let healthLoss = HEALTH_LOSS_RATE;
    if (this.airflowActive) healthLoss *= 0.5; // halve health loss during airflow
    this.health -= healthLoss * dt;
    this.wasStressed = true;
  } else if (this.water >= HEALTHY_THRESHOLD && this.light >= HEALTHY_THRESHOLD) {
    this.health += HEALTH_RECOVERY_RATE * dt;
    if (this.wasStressed && this.health > 50) {
      score += SCORE_RESTORE_BONUS * comboMultiplier;
      stats.totalRestores++;
      this.restoreFlash = 0.5;
      this.wasStressed = false;
      spawnParticle(this.x + this.w / 2, this.y + this.h / 2, COL.accent, '+' + floor(SCORE_RESTORE_BONUS *
comboMultiplier));
    }
  }

  // CHANGE #2 — Airflow grants passive health recovery while active
```

```
    if (this.airflowActive && this.health > 0 && this.health < 100) {
      this.health += AIRFLOW_HEALTH_RECOVERY * dt;
    }

    this.health = constrain(this.health, 0, 100);

    if (this.health <= 0) {
      this.isWilted = true;
      stats.plantsWilted++;
    }

    if (this.airflowActive) {
      this.airflowTimer -= dt;
      if (this.airflowTimer <= 0) {
        this.airflowActive = false;
      }
    }

    if (this.isUncertainZone) {
      this.uncertainTimer -= dt;
      if (this.uncertainTimer <= 0) {
        this.isUncertainZone = false;
      }
    }

    if (this.flashTimer > 0) this.flashTimer -= dt;
    if (this.restoreFlash > 0) this.restoreFlash -= dt;
  }

  applyWater() {
    if (this.isWilted) return;
    this.water = constrain(this.water + WATER_BOOST, 0, 100);
    this.flashTimer = 0.3;
  }

  applyLight() {
    if (this.isWilted) return;
    this.light = constrain(this.light + LIGHT_BOOST, 0, 100);
    this.flashTimer = 0.3;
  }

  applyAirflow() {
    if (this.isWilted) return;
    this.airflowActive = true;
    this.airflowTimer = AIRFLOW_DURATION;
  }
}

// ==============================================================
// PARTICLE SYSTEM
// ==============================================================
function spawnParticle(x, y, col, txt) {
  if (particles.length >= MAX_PARTICLES) particles.shift();
  particles.push({
    x, y, col: [...col],
```

```
    txt: txt || '',
    life: 1.0,
    vy: -40,
  });
}

function updateParticles(dt) {
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.life -= dt * 0.8;
    p.y += p.vy * dt;
    if (p.life <= 0) particles.splice(i, 1);
  }
}

function drawParticles() {
  for (let p of particles) {
    let a = p.life * 255;
    fill(p.col[0], p.col[1], p.col[2], a);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(14);
    textStyle(BOLD);
    text(p.txt, p.x, p.y);
  }
}

// ==============================================================
// BUTTON HELPER
// ==============================================================
class Button {
  constructor(x, y, w, h, label, callback, id) {
    this.x = x;
    this.y = y;
    this.w = w;
    this.h = h;
    this.label = label;
    this.callback = callback;
    this.id = id || label;
    this.hovered = false;
    this.visible = true;
  }

  contains(px, py) {
    return px >= this.x && px <= this.x + this.w &&
           py >= this.y && py <= this.y + this.h;
  }

  draw() {
    if (!this.visible) return;
    let col = this.hovered ? COL.buttonHover : COL.buttonBg;
    fill(col[0], col[1], col[2]);
    stroke(COL.accent[0], COL.accent[1], COL.accent[2], 120);
    strokeWeight(1.5);
    rect(this.x, this.y, this.w, this.h, 6);
```

```
    fill(COL.buttonText[0], COL.buttonText[1], COL.buttonText[2]);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(16);
    textStyle(BOLD);
    text(this.label, this.x + this.w / 2, this.y + this.h / 2);
  }

  checkHover(mx, my) {
    this.hovered = this.visible && this.contains(mx, my);
  }

  checkClick(mx, my) {
    if (this.visible && this.contains(mx, my)) {
      this.callback();
      return true;
    }
    return false;
  }
}

// ================================================================
// LAYOUT CALCULATIONS
// ================================================================
let boardX, boardY, boardW, boardH;
let bedW, bedH;
let panelX;

function computeLayout() {
  panelX = CANVAS_W - PANEL_WIDTH;
  boardX = 20;
  boardY = 20;
  boardW = panelX - 40;
  boardH = CANVAS_H - 40;

  bedW = (boardW - BED_MARGIN * (GRID_COLS + 1)) / GRID_COLS;
  bedH = (boardH - BED_MARGIN * (GRID_ROWS + 1)) / GRID_ROWS;

  for (let bed of beds) {
    bed.x = boardX + BED_MARGIN + bed.col * (bedW + BED_MARGIN);
    bed.y = boardY + BED_MARGIN + bed.row * (bedH + BED_MARGIN);
    bed.w = bedW;
    bed.h = bedH;
  }
}

// ================================================================
// INITIALIZATION
// ================================================================
function initGame() {
  beds = [];
  for (let r = 0; r < GRID_ROWS; r++) {
    for (let c = 0; c < GRID_COLS; c++) {
      beds.push(new PlantBed(c, r, r * GRID_COLS + c));
```

```
  }
 }
 computeLayout();

 selectedBed = 0;
 timer = ROUND_DURATION;
 score = 0;
 tensionMeter = 0;
 comboCount = 0;
 comboMultiplier = 1.0;
 highestCombo = 0;
 difficultyMult = 1.0;

 surgeActive = false;
 surgeTimer = 0;
 surgeNextIn = GRACE_PERIOD + random(3, 8);
 surgeCooldownLeft = 0;
 surgesCompleted = 0;
 surgeVisualIntensity = 0;

 groundingAvailable = true;
 groundingCooldownLeft = 0;
 groundingSuccessCount = 0;
 groundingStabilityLeft = 0;

 waterCooldown = 0;
 lightCooldown = 0;
 airflowCooldown = 0;
 actionLockTimer = 0;  // CHANGE #4 — reset action lock

 inputQueue = [];
 particles = [];

 stats = {
  surgesSurvived: 0,
  groundingAttempts: 0,
  groundingSuccesses: 0,
  plantsWilted: 0,
  peakCombo: 0,
  totalRestores: 0,
 };
}

// ==============================================================
// SURGE SYSTEM
// ==============================================================
function startSurge() {
 surgeActive = true;
 surgeTimer = random(SURGE_DURATION_MIN, SURGE_DURATION_MAX);
 surgeVisualIntensity = 1.0;

 let available = beds.filter(b => !b.isWilted && b.trueUrgency !== 'critical');
 shuffle(available, true);
 let count = min(FALSE_ALERT_COUNT, available.length);
 for (let i = 0; i < count; i++) {
```

```
    available[i].hasFalseAlert = true;
  }
}

function endSurge() {
  surgeActive = false;
  surgesCompleted++;
  stats.surgesSurvived++;
  score += SCORE_SURGE_SURVIVE * comboMultiplier;

  for (let b of beds) b.hasFalseAlert = false;

  let available = beds.filter(b => !b.isWilted);
  shuffle(available, true);
  let count = floor(random(UNCERTAIN_ZONE_COUNT_MIN, UNCERTAIN_ZONE_COUNT_MAX + 1));
  count = min(count, available.length);
  for (let i = 0; i < count; i++) {
    available[i].isUncertainZone = true;
    available[i].uncertainTimer = random(UNCERTAIN_DURATION_MIN, UNCERTAIN_DURATION_MAX);
  }

  let interval = lerp(SURGE_MAX_INTERVAL, SURGE_MIN_INTERVAL, (ROUND_DURATION - timer) /
ROUND_DURATION * SURGE_FREQ_RAMP);
  surgeNextIn = random(interval * 0.8, interval * 1.2);
  surgeCooldownLeft = SURGE_COOLDOWN_MIN;
}

function updateSurge(dt) {
  if (surgeActive) {
    surgeTimer -= dt;
    tensionMeter += TENSION_RISE_SURGE * dt;

    if (!reducedEffects) {
      surgeJitterX = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
      surgeJitterY = random(-SURGE_JITTER_AMOUNT, SURGE_JITTER_AMOUNT);
    }

    if (surgeTimer <= 0) {
      endSurge();
    }
  } else {
    surgeJitterX = lerp(surgeJitterX, 0, 0.2);
    surgeJitterY = lerp(surgeJitterY, 0, 0.2);

    surgeCooldownLeft -= dt;
    surgeNextIn -= dt;

    if (surgeNextIn <= 0 && surgeCooldownLeft <= 0 && timer < (ROUND_DURATION - GRACE_PERIOD)) {
      startSurge();
    }
  }

  surgeVisualIntensity = surgeActive ? lerp(surgeVisualIntensity, 1, 0.1) : lerp(surgeVisualIntensity, 0, 0.05);
}
```

```
// ==============================================================
// GROUNDING MINI-GAME
// ==============================================================
function startGrounding() {
  // CHANGE #4 — Respect action lock
  if (actionLockTimer > 0) return;
  if (!groundingAvailable || groundingCooldownLeft > 0) return;

  prevState = gameState;
  gameState = STATE.GROUNDING;
  stats.groundingAttempts++;

  groundingBeatIndex = 0;
  groundingHits = 0;
  groundingTimer = 0;
  groundingComplete = false;
  groundingCircleRadius = 0;
  groundingFeedback = '';
  groundingFeedbackTimer = 0;

  groundingBeats = [];
  for (let i = 0; i < GROUNDING_BEATS; i++) {
    groundingBeats.push({
      time: 1.0 + i * GROUNDING_BEAT_INTERVAL,
      hit: false,
      result: '',
    });
  }
}

function updateGrounding(dt) {
  groundingTimer += dt;

  // CHANGE #3 — Slower pulse rhythm using GROUNDING_BEAT_INTERVAL
  let beatPhase = (groundingTimer % GROUNDING_BEAT_INTERVAL) / GROUNDING_BEAT_INTERVAL;
  groundingCircleRadius = 30 + sin(beatPhase * PI) * 50;

  if (groundingFeedbackTimer > 0) groundingFeedbackTimer -= dt;

  if (groundingBeatIndex >= GROUNDING_BEATS) {
    if (!groundingComplete) {
      groundingComplete = true;
      groundingTimer = 0;
    }
  }

  if (groundingComplete) {
    if (groundingTimer > 1.5) {
      finishGrounding();
    }
  } else {
    if (groundingBeatIndex < GROUNDING_BEATS) {
      let beat = groundingBeats[groundingBeatIndex];
      // CHANGE #3 — Use wider window constant for miss detection
      if (groundingTimer > beat.time + GROUNDING_BEAT_WINDOW + 0.15) {
```

```
      beat.result = 'missed';
      groundingFeedback = 'Missed';
      groundingFeedbackTimer = 0.6;
      groundingBeatIndex++;
    }
  }
}

  // Grounding fully pauses plant decay (player isn't punished for coping)
  // (no bed.update call here)
}

function groundingInput() {
  if (groundingComplete || groundingBeatIndex >= GROUNDING_BEATS) return;

  let beat = groundingBeats[groundingBeatIndex];
  let diff = abs(groundingTimer - beat.time);

  // CHANGE #3 — Uses wider GROUNDING_BEAT_WINDOW for easier hits
  if (diff <= GROUNDING_BEAT_WINDOW) {
    beat.hit = true;
    beat.result = 'hit';
    groundingHits++;
    // CHANGE #3 — Extra positive feedback for timing quality
    if (diff <= GROUNDING_BEAT_WINDOW * 0.4) {
      groundingFeedback = 'Perfect!';
    } else {
      groundingFeedback = 'Steady!';
    }
    groundingFeedbackTimer = 0.5;
  } else if (groundingTimer < beat.time - GROUNDING_BEAT_WINDOW) {
    beat.result = 'early';
    groundingFeedback = 'Too Early';
    groundingFeedbackTimer = 0.5;
  } else {
    beat.result = 'late';
    groundingFeedback = 'Too Late';
    groundingFeedbackTimer = 0.5;
  }
  groundingBeatIndex++;
}

function finishGrounding() {
  // CHANGE #3 — Use configurable success threshold (default 1/3 hits)
  let success = groundingHits >= GROUNDING_SUCCESS_THRESHOLD;

  if (success) {
    tensionMeter = max(0, tensionMeter - GROUNDING_TENSION_REWARD);
    groundingStabilityLeft = GROUNDING_STABILITY_TIME;
    groundingSuccessCount++;
    stats.groundingSuccesses++;
    score += SCORE_GROUNDING_BONUS * comboMultiplier;

    for (let b of beds) b.hasFalseAlert = false;
  } else {
```

```
    tensionMeter = max(0, tensionMeter - GROUNDING_PARTIAL_REWARD);
  }

  groundingCooldownLeft = GROUNDING_COOLDOWN;
  gameState = STATE.PLAYING;
}

// ==============================================================
// TENSION METER
// ==============================================================
function updateTension(dt) {
  let activeAlerts = beds.filter(b => !b.isWilted && b.trueUrgency === 'critical').length;
  tensionMeter += activeAlerts * TENSION_RISE_ALERTS * dt * 0.3;

  if (!surgeActive && activeAlerts < 2) {
    tensionMeter -= TENSION_DECAY * dt;
  }

  if (groundingStabilityLeft > 0) {
    tensionMeter -= TENSION_DECAY * 2 * dt;
    groundingStabilityLeft -= dt;
  }

  tensionMeter = constrain(tensionMeter, 0, 100);

  if (tensionMeter >= 100) {
    tensionMeter = TENSION_OVERLOAD_RESET;
    comboMultiplier = max(1.0, comboMultiplier - 0.5);
  }
}

// ==============================================================
// COMBO SYSTEM
// ==============================================================
function updateCombo(dt) {
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40 && b.water > 30 && b.light > 30).length;

  if (healthyCount >= COMBO_THRESHOLD) {
    comboCount++;
    comboMultiplier = min(COMBO_MAX_MULTIPLIER, 1.0 + floor(comboCount / 60) *
COMBO_MULTIPLIER_STEP);
  } else {
    if (comboCount > 0) {
      comboCount = max(0, comboCount - 3);
      if (comboCount === 0) comboMultiplier = 1.0;
    }
  }

  highestCombo = max(highestCombo, comboMultiplier);
  stats.peakCombo = highestCombo;
}

function updateScore(dt) {
  let healthyCount = beds.filter(b => !b.isWilted && b.health > 40).length;
  score += healthyCount * SCORE_PER_HEALTHY_SEC * comboMultiplier * dt;
```

```
}

function updateDifficulty() {
  let elapsed = ROUND_DURATION - timer;
  let progress = elapsed / ROUND_DURATION;
  difficultyMult = 1.0 + progress * DRAIN_RAMP_FACTOR;
}

// ==============================================================
// ACTION HANDLING
// ==============================================================
function doAction(type) {
  // CHANGE #4 — Block all actions during action lock
  if (actionLockTimer > 0) return;

  let bed = beds[selectedBed];
  if (!bed || bed.isWilted) return;

  if (bed.isUncertainZone) {
    tensionMeter += TENSION_RISE_UNCERTAIN;
  }

  if (type === 'water' && waterCooldown <= 0) {
    bed.applyWater();
    waterCooldown = ACTION_COOLDOWN;
    actionLockTimer = ACTION_LOCK_DURATION;  // CHANGE #4 — trigger global lock
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.water, 'Water');
  } else if (type === 'light' && lightCooldown <= 0) {
    bed.applyLight();
    lightCooldown = ACTION_COOLDOWN;
    actionLockTimer = ACTION_LOCK_DURATION;  // CHANGE #4 — trigger global lock
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.light, 'Light');
  } else if (type === 'airflow' && airflowCooldown <= 0) {
    bed.applyAirflow();
    airflowCooldown = AIRFLOW_COOLDOWN;
    // CHANGE #4 — No action lock for airflow (already has long cooldown)
    spawnParticle(bed.x + bed.w / 2, bed.y + 10, COL.airflow, 'Airflow');
  }
}

function processInputQueue(dt) {
  for (let i = inputQueue.length - 1; i >= 0; i--) {
    inputQueue[i].delay -= dt;
    if (inputQueue[i].delay <= 0) {
      inputQueue[i].action();
      inputQueue.splice(i, 1);
    }
  }
}

function queueAction(fn) {
  if (surgeActive) {
    inputQueue.push({ action: fn, delay: SURGE_INPUT_DELAY });
  } else {
    fn();
```

```
  }
}

// ==============================================================
// CHECK WIN / LOSE
// ==============================================================
function checkEndConditions() {
  let wiltedCount = beds.filter(b => b.isWilted).length;

  if (wiltedCount >= LOSE_WILTED_COUNT) {
    gameState = STATE.LOSE;
    return;
  }

  if (timer <= 0) {
    let aliveCount = beds.filter(b => !b.isWilted).length;
    if (aliveCount >= WIN_MIN_ALIVE) {
      gameState = STATE.WIN;
    } else {
      gameState = STATE.LOSE;
    }
  }
}

function computeGrade() {
  let maxPossible = TOTAL_BEDS * SCORE_PER_HEALTHY_SEC * ROUND_DURATION * 2;
  let ratio = score / maxPossible;
  if (ratio > 0.6) return 'A';
  if (ratio > 0.4) return 'B';
  if (ratio > 0.25) return 'C';
  return 'D';
}

function getBedAtMouse(mx, my) {
  for (let i = 0; i < beds.length; i++) {
    let b = beds[i];
    if (mx >= b.x && mx <= b.x + b.w && my >= b.y && my <= b.y + b.h) {
      return i;
    }
  }
  return -1;
}

// ==============================================================
// DRAW FUNCTIONS
// ==============================================================

function drawTitle() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let t = millis() / 1000;
  noStroke();
  for (let i = 0; i < 8; i++) {
    let x = CANVAS_W / 2 + cos(t * 0.3 + i * 0.8) * 200;
    let y = CANVAS_H / 2 + sin(t * 0.4 + i * 1.1) * 120;
```

```
    let r = 40 + sin(t * 0.5 + i) * 15;
    fill(COL.accent[0], COL.accent[1], COL.accent[2], 15 + sin(t + i) * 8);
    ellipse(x, y, r * 2, r * 2);
  }

  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(52);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Garden Circuit', CANVAS_W / 2, CANVAS_H / 2 - 120);

  textStyle(NORMAL);
  textSize(18);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Keep the greenhouse in balance', CANVAS_W / 2, CANVAS_H / 2 - 70);

  stroke(COL.accent[0], COL.accent[1], COL.accent[2], 60);
  strokeWeight(1);
  line(CANVAS_W / 2 - 120, CANVAS_H / 2 - 45, CANVAS_W / 2 + 120, CANVAS_H / 2 - 45);
  noStroke();
}

function drawInstructions() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  let cx = CANVAS_W / 2;
  let startY = 22;
  let maxTextY = CANVAS_H - 80;

  textAlign(CENTER, TOP);
  textStyle(BOLD);
  textSize(28);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('How to Play', cx, startY);

  textStyle(NORMAL);
  textSize(13);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  textAlign(LEFT, TOP);

  let leftX = 40;
  let rightX = CANVAS_W / 2 + 30;
  let lineH = 18;

  let y = startY + 45;

  let leftLines = [
    { t: '[ OBJECTIVE ]', style: 'header' },
    { t: 'Keep 9 plant beds healthy for 90 seconds.', style: 'normal' },
    { t: 'Plants need Water and Light — both drain over time.', style: 'normal' },
    { t: 'If needs drop too low, health decreases. Lose 3 plants = game over.', style: 'normal' },
    { t: '', style: 'normal' },
    { t: '[ CONTROLS ]', style: 'header' },
    { t: 'Arrow Keys / WASD — Move selection cursor', style: 'normal' },
    { t: 'Click a bed — Select it directly', style: 'normal' },
```

```
  { t: 'Q  or  Water btn — Route water to selected bed', style: 'normal' },
  { t: 'E  or  Light btn — Route light to selected bed', style: 'normal' },
  { t: 'R  or  Airflow btn — Temporary stabilizer (cooldown)', style: 'normal' },
  { t: 'Space — Begin Grounding Routine (when ready)', style: 'normal' },
  { t: 'P — Pause / Resume     V — Toggle reduced effects', style: 'sub' },
  { t: '', style: 'normal' },
  { t: '[ SCORING & HARMONY ]', style: 'header' },
  { t: 'Earn points by keeping beds healthy over time.', style: 'normal' },
  { t: 'Restore critical plants for bonus points.', style: 'normal' },
  { t: 'Keep 6+ beds healthy to build a Harmony combo.', style: 'normal' },
  { t: 'Higher combos multiply your score!', style: 'normal' },
];

for (let line of leftLines) {
  if (line.style === 'header') {
    fill(COL.accent[0], COL.accent[1], COL.accent[2]);
    textStyle(BOLD);
    textSize(13);
  } else if (line.style === 'sub') {
    fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
    textStyle(NORMAL);
    textSize(12);
  } else {
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
    textStyle(NORMAL);
    textSize(13);
  }
  text(line.t, leftX, y);
  y += lineH;
}

y = startY + 45;

let rightLines = [
  { t: '[ SURGES — System Disruptions ]', style: 'header' },
  { t: 'At unpredictable intervals, a Surge disrupts the', style: 'normal' },
  { t: 'greenhouse for a few seconds:', style: 'normal' },
  { t: '  · Some indicators flicker with false urgency alerts', style: 'sub' },
  { t: '  · Visual clutter increases briefly', style: 'sub' },
  { t: '  · Controls have a slight delay', style: 'sub' },
  { t: 'Stay focused and adapt — surges are manageable.', style: 'normal' },
  { t: '', style: 'normal' },
  { t: '[ UNCERTAIN ZONES — Post-Surge ]', style: 'header' },
  { t: 'After a surge, some beds become Uncertain Zones:', style: 'normal' },
  { t: '  · Interacting with them raises Tension faster', style: 'sub' },
  { t: '  · Avoiding them may leave plants under-served', style: 'sub' },
  { t: 'Balance efficiency against rising tension.', style: 'normal' },
  { t: '', style: 'normal' },
  { t: '[ GROUNDING — Coping & Recovery ]', style: 'header' },
  { t: 'Press Space to start a Grounding Routine:', style: 'normal' },
  { t: '  · Match 3 slow pulse beats by pressing Space in rhythm', style: 'sub' },
  { t: '  · Success: reduces Tension, clears false alerts,', style: 'sub' },
  { t: '    and stabilizes the garden briefly', style: 'sub' },
  { t: '  · Has a cooldown — use it strategically', style: 'sub' },
  { t: '', style: 'normal' },
```

```
    { t: '[ A NOTE ON DESIGN ]', style: 'header' },
    { t: 'This game uses symbolic mechanics — surges, uncertainty,', style: 'normal' },
    { t: 'and grounding — to respectfully represent themes of', style: 'normal' },
    { t: 'disruption and coping. It is not a literal simulation.', style: 'normal' },
  ];

  for (let line of rightLines) {
    if (line.style === 'header') {
      fill(COL.accent[0], COL.accent[1], COL.accent[2]);
      textStyle(BOLD);
      textSize(13);
    } else if (line.style === 'sub') {
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
      textStyle(NORMAL);
      textSize(12);
    } else {
      fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
      textStyle(NORMAL);
      textSize(13);
    }
    text(line.t, rightX, y);
    y += lineH;
  }

  stroke(COL.accent[0], COL.accent[1], COL.accent[2], 40);
  strokeWeight(1);
  line(CANVAS_W / 2 + 10, startY + 50, CANVAS_W / 2 + 10, maxTextY - 10);
  noStroke();
}

// --- Gameplay Drawing ---
function drawGameplay() {
  if (!reducedEffects && surgeVisualIntensity > 0.1) {
    translate(surgeJitterX * surgeVisualIntensity, surgeJitterY * surgeVisualIntensity);
  }

  background(COL.bg[0], COL.bg[1], COL.bg[2]);

  if (surgeActive && !reducedEffects) {
    noStroke();
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 15 + sin(millis() / 80) * 10);
    rect(0, 0, CANVAS_W, CANVAS_H);

    for (let i = 0; i < 3; i++) {
      let fx = random(boardX, boardX + boardW);
      let fy = random(boardY, boardY + boardH);
      fill(COL.surge[0], COL.surge[1], COL.surge[2], 30);
      textSize(12);
      textAlign(CENTER, CENTER);
      text('!', fx, fy);
    }
  }

  for (let bed of beds) {
    drawBed(bed);
```

```
  }

  if (beds[selectedBed]) {
    let sb = beds[selectedBed];
    noFill();
    stroke(COL.accent[0], COL.accent[1], COL.accent[2], 180 + sin(millis() / 200) * 60);
    strokeWeight(3);
    rect(sb.x - 3, sb.y - 3, sb.w + 6, sb.h + 6, 6);
    noStroke();
  }

  drawParticles();
  drawPanel();

  if (surgeActive) {
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 180 + sin(millis() / 100) * 60);
    textAlign(CENTER, CENTER);
    textSize(20);
    textStyle(BOLD);
    text('⚡ SURGE ACTIVE ⚡', boardX + boardW / 2, boardY - 3);
    textStyle(NORMAL);
  }

  // CHANGE #4 — Subtle action lock overlay on the board area
  if (actionLockTimer > 0) {
    noStroke();
    fill(COL.bg[0], COL.bg[1], COL.bg[2], 30);
    rect(boardX, boardY, boardW, boardH);
  }
}

// ==============================================================
// CHANGE #1 — Rewritten drawBed() with plant images
// ==============================================================
function drawBed(bed) {
  let bx = bed.x;
  let by = bed.y;
  let bw = bed.w;
  let bh = bed.h;

  // --- Background rectangle (always drawn as base) ---
  if (bed.isWilted) {
    fill(COL.bedDead[0], COL.bedDead[1], COL.bedDead[2]);
  } else if (bed.isUncertainZone) {
    let shimmer = sin(millis() / 300 + bed.index) * 10;
    fill(COL.bedUncertain[0] + shimmer, COL.bedUncertain[1] + shimmer, COL.bedUncertain[2] + shimmer);
  } else {
    fill(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]);
  }

  if (bed.flashTimer > 0) {
    let flashAlpha = bed.flashTimer / 0.3;
    fill(lerpColor(
      color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
      color(COL.accent[0], COL.accent[1], COL.accent[2]),
```

```
    flashAlpha * 0.3
  ));
}

if (bed.restoreFlash > 0) {
  let flashAlpha = bed.restoreFlash / 0.5;
  fill(lerpColor(
    color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
    color(100, 255, 150),
    flashAlpha * 0.4
  ));
}

stroke(60, 70, 60);
strokeWeight(1);
rect(bx, by, bw, bh, 4);
noStroke();

// --- CHANGE #1: Draw plant image OR fallback icon ---
if (imagesLoaded) {
  // Pick the correct image based on health thresholds
  let img = getPlantImage(bed.health);
  if (img) {
    // Draw image centered in the upper portion of the bed
    let imgSize = min(bw - 20, bh * 0.4);  // fit within bed
    let imgX = bx + (bw - imgSize) / 2;
    let imgY = by + 6;

    push();
    imageMode(CORNER);
    // If dead, add dark tint overlay for extra clarity
    if (bed.isWilted) {
      tint(100, 60, 60, 180); // desaturated reddish tint
    }
    image(img, imgX, imgY, imgSize, imgSize);
    noTint();
    pop();
  }
} else {
  // Fallback: original text icon rendering (no images available)
  if (bed.isWilted) {
    fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 120);
    textAlign(CENTER, CENTER);
    textSize(28);
    text('✕', bx + bw / 2, by + bh / 2 - 10);
    textSize(12);
    text('Wilted', bx + bw / 2, by + bh / 2 + 18);
    return;
  }

  let healthColor = bed.health > 50 ?
    COL.health :
    [lerp(COL.healthLow[0], COL.health[0], bed.health / 50),
     lerp(COL.healthLow[1], COL.health[1], bed.health / 50),
     lerp(COL.healthLow[2], COL.health[2], bed.health / 50)];
```

```
  fill(healthColor[0], healthColor[1], healthColor[2]);
  textAlign(CENTER, CENTER);
  textSize(22);
  let plantIcon = bed.health > 60 ? '❋' : bed.health > 30 ? '❊' : '✿';
  text(plantIcon, bx + bw / 2, by + 24);
}

// If wilted and images loaded, still show "Wilted" label and skip meters
if (bed.isWilted) {
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 180);
  textAlign(CENTER, CENTER);
  textSize(12);
  textStyle(BOLD);
  text('Wilted', bx + bw / 2, by + bh - 16);
  textStyle(NORMAL);
  return;
}

// --- UI overlays (meters, labels, alerts) — drawn ON TOP of image ---

// Health bar
let barStartY = by + bh * 0.45;  // position bars below the plant image area
let barH = 7;
let barW = bw - 20;
let barX = bx + 10;

let healthColor = bed.health > 50 ?
  COL.health :
  [lerp(COL.healthLow[0], COL.health[0], bed.health / 50),
   lerp(COL.healthLow[1], COL.health[1], bed.health / 50),
   lerp(COL.healthLow[2], COL.health[2], bed.health / 50)];

fill(30, 35, 30);
rect(barX, barStartY, barW, barH, 2);
let hw = barW * bed.health / 100;
fill(healthColor[0], healthColor[1], healthColor[2]);
rect(barX, barStartY, hw, barH, 2);

textSize(9);
textStyle(BOLD);
fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
textAlign(CENTER, TOP);
text('HP ' + floor(bed.health), bx + bw / 2, barStartY + barH + 2);
textStyle(NORMAL);

// Water bar
let wBarY = barStartY + barH + 16;
fill(30, 35, 50);
rect(barX, wBarY, barW, barH, 2);
fill(COL.water[0], COL.water[1], COL.water[2]);
rect(barX, wBarY, barW * bed.water / 100, barH, 2);

textSize(9);
fill(COL.water[0], COL.water[1], COL.water[2]);
```

```
  textAlign(LEFT, TOP);
  text('💧' + floor(bed.water), barX, wBarY + barH + 2);

  // Light bar
  let lBarY = wBarY + barH + 16;
  fill(30, 35, 30);
  rect(barX, lBarY, barW, barH, 2);
  fill(COL.light[0], COL.light[1], COL.light[2]);
  rect(barX, lBarY, barW * bed.light / 100, barH, 2);

  fill(COL.light[0], COL.light[1], COL.light[2]);
  textAlign(LEFT, TOP);
  text('☀' + floor(bed.light), barX, lBarY + barH + 2);

  // Airflow indicator
  if (bed.airflowActive) {
    fill(COL.airflow[0], COL.airflow[1], COL.airflow[2], 160);
    textAlign(RIGHT, TOP);
    text('💨' + floor(bed.airflowTimer) + 's', barX + barW, lBarY + barH + 2);
  }

  // Surge flicker overlay
  if (surgeActive && !reducedEffects && random() < 0.15) {
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 50);
    noStroke();
    rect(bx, by, bw, bh, 4);
  }

  // False alert / urgency indicators
  if (bed.hasFalseAlert && !bed.isWilted) {
    fill(COL.falseAlert[0], COL.falseAlert[1], COL.falseAlert[2], 160 + sin(millis() / 150) * 60);
    textAlign(RIGHT, TOP);
    textSize(16);
    text('⚠', bx + bw - 4, by + 4);
  } else if (bed.displayUrgency === 'critical') {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2], 180);
    textAlign(RIGHT, TOP);
    textSize(14);
    text('!!', bx + bw - 6, by + 4);
  } else if (bed.displayUrgency === 'warning') {
    fill(COL.light[0], COL.light[1], COL.light[2], 140);
    textAlign(RIGHT, TOP);
    textSize(13);
    text('!', bx + bw - 6, by + 6);
  }

  // Uncertain zone border + label
  if (bed.isUncertainZone) {
    noFill();
    stroke(COL.bedUncertain[0] + 40, COL.bedUncertain[1] + 40, COL.bedUncertain[2] + 40, 100 + sin(millis() /
400 + bed.index) * 50);
    strokeWeight(2);
    rect(bx + 2, by + 2, bw - 4, bh - 4, 3);
    noStroke();
```

```
    fill(180, 160, 220, 140);
    textAlign(LEFT, BOTTOM);
    textSize(8);
    text('uncertain', bx + 5, by + bh - 3);
  }

  // Bed index label
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 60);
  textAlign(LEFT, TOP);
  textSize(8);
  text('#' + (bed.index + 1), bx + 4, by + 4);
}

// --- Side Panel ---
function drawPanel() {
  fill(COL.panelBg[0], COL.panelBg[1], COL.panelBg[2]);
  stroke(50, 60, 50);
  strokeWeight(1);
  rect(panelX, 0, PANEL_WIDTH, CANVAS_H);
  noStroke();

  let px = panelX + 20;
  let py = 20;
  let pw = PANEL_WIDTH - 40;

  textAlign(LEFT, TOP);
  textStyle(BOLD);
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('TIME', px, py);
  textSize(30);
  fill(timer < 15 ? COL.healthLow[0] : COL.textPrimary[0],
       timer < 15 ? COL.healthLow[1] : COL.textPrimary[1],
       timer < 15 ? COL.healthLow[2] : COL.textPrimary[2]);
  text(floor(timer) + 's', px, py + 16);
  py += 60;

  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('SCORE', px, py);
  textSize(24);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text(floor(score), px, py + 16);
  py += 52;

  textSize(13);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text('HARMONY  x' + nf(comboMultiplier, 1, 1), px, py);
  py += 20;
  fill(30, 30, 20);
  rect(px, py, pw, 8, 3);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  rect(px, py, pw * ((comboMultiplier - 1) / (COMBO_MAX_MULTIPLIER - 1)), 8, 3);
  py += 24;
```

```
textSize(13);
fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
text('TENSION', px, py);
py += 18;
fill(30, 25, 25);
rect(px, py, pw, 14, 4);
let tensionCol = tensionMeter < 30 ? [80, 180, 120] :
            tensionMeter < 60 ? [200, 180, 60] :
            tensionMeter < 80 ? [220, 120, 50] :
            [200, 60, 60];
fill(tensionCol[0], tensionCol[1], tensionCol[2]);
rect(px, py, pw * tensionMeter / 100, 14, 4);
fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
textAlign(CENTER, CENTER);
textSize(10);
text(floor(tensionMeter), px + pw / 2, py + 7);
textAlign(LEFT, TOP);
py += 30;

textSize(13);
if (surgeActive) {
  fill(COL.surge[0], COL.surge[1], COL.surge[2]);
  textStyle(BOLD);
  text('⚡ SURGE ACTIVE', px, py);
  textStyle(NORMAL);
} else {
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('System Stable', px, py);
}
py += 28;

textSize(13);
fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
text('GROUNDING', px, py);
py += 18;
if (groundingCooldownLeft > 0) {
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  textSize(12);
  text('Cooldown: ' + floor(groundingCooldownLeft) + 's', px, py);
  py += 16;
  fill(30, 30, 30);
  rect(px, py, pw, 8, 3);
  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2], 120);
  rect(px, py, pw * (1 - groundingCooldownLeft / GROUNDING_COOLDOWN), 8, 3);
} else {
  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
  textSize(13);
  textStyle(BOLD);
  text('Ready [SPACE]', px, py);
  textStyle(NORMAL);
  py += 16;
}
py += 24;
```

```
  stroke(60, 70, 60, 80);
  strokeWeight(1);
  line(px, py, px + pw, py);
  noStroke();
  py += 14;

  let sb = beds[selectedBed];
  textSize(13);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('SELECTED BED #' + (selectedBed + 1), px, py);
  py += 20;

  if (sb && !sb.isWilted) {
    textSize(12);
    fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
    text('Health: ' + floor(sb.health), px, py); py += 16;
    fill(COL.water[0], COL.water[1], COL.water[2]);
    text('Water:  ' + floor(sb.water), px, py); py += 16;
    fill(COL.light[0], COL.light[1], COL.light[2]);
    text('Light:  ' + floor(sb.light), px, py); py += 16;
    if (sb.airflowActive) {
      fill(COL.airflow[0], COL.airflow[1], COL.airflow[2]);
      text('Airflow: ' + nf(sb.airflowTimer, 1, 1) + 's', px, py);
    }
    if (sb.isUncertainZone) {
      fill(180, 160, 220);
      py += 16;
      text('⬡ Uncertain Zone', px, py);
    }
  } else if (sb && sb.isWilted) {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2]);
    textSize(12);
    text('This plant has wilted.', px, py);
  }
  py += 30;

  drawActionButtons(px, py, pw);

  let wiltedCount = beds.filter(b => b.isWilted).length;
  textAlign(LEFT, BOTTOM);
  textSize(12);
  fill(wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[0] : COL.textSecondary[0],
       wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[1] : COL.textSecondary[1],
       wiltedCount >= LOSE_WILTED_COUNT - 1 ? COL.healthLow[2] : COL.textSecondary[2]);
  text('Wilted: ' + wiltedCount + ' / ' + LOSE_WILTED_COUNT + ' max', px, CANVAS_H - 38);

  textSize(10);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 100);
  text('P=Pause  V=Effects', px, CANVAS_H - 18);
}

// --- Action Buttons ---
let actionBtns = [];

function drawActionButtons(px, py, pw) {
```

```
  let btnW = pw;
  let btnH = 32;
  let gap = 8;

  if (actionBtns.length === 0) {
    actionBtns.push(new Button(px, py, btnW, btnH, '💧 Water  [Q]', () => queueAction(() =>
doAction('water')), 'water'));
    actionBtns.push(new Button(px, py + btnH + gap, btnW, btnH, '☀ Light  [E]', () => queueAction(() =>
doAction('light')), 'light'));
    actionBtns.push(new Button(px, py + 2 * (btnH + gap), btnW, btnH, '🌬 Airflow  [R]', () => queueAction(()
=> doAction('airflow')), 'airflow'));
    actionBtns.push(new Button(px, py + 3 * (btnH + gap), btnW, btnH, '◉ Ground  [SPACE]', () =>
startGrounding(), 'ground'));
  }

  actionBtns[0].y = py;
  actionBtns[1].y = py + btnH + gap;
  actionBtns[2].y = py + 2 * (btnH + gap);
  actionBtns[3].y = py + 3 * (btnH + gap);

  for (let btn of actionBtns) {
    btn.checkHover(mouseX, mouseY);

    let available = true;
    if (btn.id === 'water') available = waterCooldown <= 0;
    if (btn.id === 'light') available = lightCooldown <= 0;
    if (btn.id === 'airflow') available = airflowCooldown <= 0;
    if (btn.id === 'ground') available = groundingCooldownLeft <= 0 && gameState === STATE.PLAYING;

    // CHANGE #4 — Action lock dims ALL action buttons
    if (actionLockTimer > 0) available = false;

    if (!available) {
      fill(35, 40, 38);
      stroke(50, 55, 50);
      strokeWeight(1);
      rect(btn.x, btn.y, btn.w, btn.h, 6);
      noStroke();
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 100);
      textAlign(CENTER, CENTER);
      textSize(14);
      text(btn.label, btn.x + btn.w / 2, btn.y + btn.h / 2);
    } else {
      btn.draw();
    }
  }
}

// --- Grounding Mini-Game Screen ---
function drawGrounding() {
  fill(COL.bg[0], COL.bg[1], COL.bg[2], 200);
  rect(0, 0, CANVAS_W, CANVAS_H);

  let cx = CANVAS_W / 2;
```

```
let cy = CANVAS_H / 2;

fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
textAlign(CENTER, CENTER);
textStyle(BOLD);
textSize(24);
text('Grounding Routine', cx, cy - 140);
textStyle(NORMAL);

// CHANGE #3 — Clearer instructions with hint about timing
textSize(14);
fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
text('Press SPACE when the circle is largest', cx, cy - 110);
textSize(11);
fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 150);
text('(generous timing — just match the rhythm)', cx, cy - 92);

let beatPhase;
if (groundingBeatIndex < GROUNDING_BEATS && !groundingComplete) {
  let currentBeatTime = groundingBeats[groundingBeatIndex].time;
  let distToBeat = currentBeatTime - groundingTimer;
  beatPhase = 1.0 - constrain(abs(distToBeat) / (GROUNDING_BEAT_INTERVAL * 0.5), 0, 1);
} else {
  beatPhase = 0.5;
}

let radius = 30 + beatPhase * 60;

noFill();
stroke(COL.grounding[0], COL.grounding[1], COL.grounding[2], 80);
strokeWeight(2);
ellipse(cx, cy, 180, 180);

// CHANGE #3 — Show timing window ring to help players see when to press
if (groundingBeatIndex < GROUNDING_BEATS && !groundingComplete) {
  stroke(COL.grounding[0], COL.grounding[1], COL.grounding[2], 40);
  strokeWeight(1);
  ellipse(cx, cy, 160, 160);  // inner "safe zone" ring
}

let alpha = 120 + beatPhase * 135;
fill(COL.grounding[0], COL.grounding[1], COL.grounding[2], alpha);
stroke(COL.grounding[0], COL.grounding[1], COL.grounding[2], 200);
strokeWeight(2);
ellipse(cx, cy, radius * 2, radius * 2);
noStroke();

for (let i = 0; i < GROUNDING_BEATS; i++) {
  let bx = cx - 40 + i * 40;
  let by = cy + 120;

  if (groundingBeats[i] && groundingBeats[i].result === 'hit') {
    fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
  } else if (groundingBeats[i] && (groundingBeats[i].result === 'early' || groundingBeats[i].result === 'late' ||
groundingBeats[i].result === 'missed')) {
```

```
      fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2], 150);
    } else if (i === groundingBeatIndex) {
      fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2], 180 + sin(millis() / 200) * 60);
    } else {
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 80);
    }
    ellipse(bx, by, 20, 20);
  }

  // CHANGE #3 — Show hit count so player knows progress
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 140);
  textSize(11);
  text(groundingHits + '/' + GROUNDING_SUCCESS_THRESHOLD + ' needed', cx, cy + 150);

  if (groundingFeedbackTimer > 0 && groundingFeedback) {
    // CHANGE #3 — "Perfect!" gets a distinct gold color
    let feedCol;
    if (groundingFeedback === 'Perfect!') {
      feedCol = COL.combo;
    } else if (groundingFeedback === 'Steady!') {
      feedCol = COL.grounding;
    } else {
      feedCol = COL.falseAlert;
    }
    fill(feedCol[0], feedCol[1], feedCol[2], groundingFeedbackTimer / 0.6 * 255);
    textSize(20);
    textStyle(BOLD);
    text(groundingFeedback, cx, cy + 80);
    textStyle(NORMAL);
  }

  if (groundingComplete) {
    let success = groundingHits >= GROUNDING_SUCCESS_THRESHOLD;
    if (success) {
      fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
      textSize(22);
      textStyle(BOLD);
      text('Grounded — Clarity Restored', cx, cy - 170);
    } else {
      fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
      textSize(18);
      text('Partial focus — keep going', cx, cy - 170);
    }
    textStyle(NORMAL);
  }
}

// --- Win/Lose/Pause screens (unchanged) ---
function drawWin() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);
  let cx = CANVAS_W / 2;
  let t = millis() / 1000;
  noStroke();
  for (let i = 0; i < 12; i++) {
    let x = cx + cos(t * 0.2 + i * 0.5) * 300;
```

```
    let y = CANVAS_H / 2 + sin(t * 0.3 + i * 0.7) * 200;
    fill(COL.accent[0], COL.accent[1], COL.accent[2], 12);
    ellipse(x, y, 60, 60);
  }
  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(42);
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Greenhouse Stabilized', cx, 100);
  textStyle(NORMAL);
  textSize(18);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('You kept the garden in balance.', cx, 150);
  let grade = computeGrade();
  textSize(60);
  textStyle(BOLD);
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text(grade, cx, 230);
  textStyle(NORMAL);
  drawEndStats(cx, 290);
}

function drawLose() {
  background(COL.bg[0], COL.bg[1], COL.bg[2]);
  let cx = CANVAS_W / 2;
  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(36);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('The Garden Faded', cx, 100);
  textStyle(NORMAL);
  textSize(16);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Too many plants were lost — but every attempt is practice.', cx, 145);
  text('Adjust your strategy and try again.', cx, 168);
  drawEndStats(cx, 230);
}

function drawEndStats(cx, startY) {
  let y = startY;
  let lineH = 30;
  textSize(16);
  textAlign(CENTER, CENTER);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('Score: ' + floor(score), cx, y); y += lineH;
  fill(COL.combo[0], COL.combo[1], COL.combo[2]);
  text('Best Harmony: x' + nf(stats.peakCombo, 1, 1), cx, y); y += lineH;
  fill(COL.surge[0], COL.surge[1], COL.surge[2]);
  text('Surges Survived: ' + stats.surgesSurvived, cx, y); y += lineH;
  fill(COL.grounding[0], COL.grounding[1], COL.grounding[2]);
  text('Grounding: ' + stats.groundingSuccesses + ' / ' + stats.groundingAttempts + ' successful', cx, y); y +=
lineH;
  fill(COL.accent[0], COL.accent[1], COL.accent[2]);
  text('Plants Restored: ' + stats.totalRestores, cx, y); y += lineH;
  fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2]);
```

```
  text('Plants Wilted: ' + stats.plantsWilted, cx, y);
}

function drawPause() {
  fill(COL.bg[0], COL.bg[1], COL.bg[2], 180);
  rect(0, 0, CANVAS_W, CANVAS_H);
  textAlign(CENTER, CENTER);
  textStyle(BOLD);
  textSize(36);
  fill(COL.textPrimary[0], COL.textPrimary[1], COL.textPrimary[2]);
  text('PAUSED', CANVAS_W / 2, CANVAS_H / 2 - 30);
  textStyle(NORMAL);
  textSize(16);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2]);
  text('Press P to resume', CANVAS_W / 2, CANVAS_H / 2 + 20);
}

// ==============================================================
// P5.JS SETUP
// ==============================================================
function setup() {
  createCanvas(CANVAS_W, CANVAS_H);
  textFont('monospace');
  checkImagesLoaded();  // CHANGE #1 — verify images after preload
  initGame();
  initMenuButtons();
}

// ==============================================================
// MENU BUTTONS
// ==============================================================
let menuButtons = [];

function initMenuButtons() {
  menuButtons = [];
  let cx = CANVAS_W / 2;
  let btnW = 200;
  let btnH = 48;

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H / 2 - 10, btnW, btnH, 'Start Game', () => {
    initGame();
    gameState = STATE.PLAYING;
  }, 'title_start'));

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H / 2 + 55, btnW, btnH, 'Instructions', () => {
    gameState = STATE.INSTRUCTIONS;
  }, 'title_instructions'));

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 55, btnW, btnH, 'Back to Title', () => {
    gameState = STATE.TITLE;
  }, 'instr_back'));

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 120, btnW, btnH, 'Play Again', () => {
    actionBtns = [];
    initGame();
```

```
    gameState = STATE.PLAYING;
  }, 'end_restart'));

  menuButtons.push(new Button(cx - btnW / 2, CANVAS_H - 60, btnW, btnH, 'Title Screen', () => {
    actionBtns = [];
    gameState = STATE.TITLE;
  }, 'end_title'));
}

// ==============================================================
// DRAW LOOP
// ==============================================================
function draw() {
  let dt = deltaTime / 1000;
  dt = min(dt, 0.05);

  for (let btn of menuButtons) {
    btn.visible = false;
    btn.checkHover(mouseX, mouseY);
  }

  switch (gameState) {
    case STATE.TITLE:
      drawTitle();
      showButton('title_start');
      showButton('title_instructions');
      break;
    case STATE.INSTRUCTIONS:
      drawInstructions();
      showButton('instr_back');
      break;
    case STATE.PLAYING:
      updateGame(dt);
      drawGameplay();
      break;
    case STATE.GROUNDING:
      drawGameplay();
      updateGrounding(dt);
      drawGrounding();
      break;
    case STATE.WIN:
      drawWin();
      showButton('end_restart');
      showButton('end_title');
      break;
    case STATE.LOSE:
      drawLose();
      showButton('end_restart');
      showButton('end_title');
      break;
    case STATE.PAUSED:
      drawGameplay();
      drawPause();
      break;
  }
```

```
  for (let btn of menuButtons) {
    if (btn.visible) btn.draw();
  }
}

function showButton(id) {
  for (let btn of menuButtons) {
    if (btn.id === id) {
      btn.visible = true;
      btn.checkHover(mouseX, mouseY);
    }
  }
}

// ==============================================================
// MAIN UPDATE
// ==============================================================
function updateGame(dt) {
  timer -= dt;
  timer = max(0, timer);

  updateDifficulty();

  if (waterCooldown > 0) waterCooldown -= dt;
  if (lightCooldown > 0) lightCooldown -= dt;
  if (airflowCooldown > 0) airflowCooldown -= dt;
  if (groundingCooldownLeft > 0) groundingCooldownLeft -= dt;

  // CHANGE #4 — Tick down action lock timer
  if (actionLockTimer > 0) actionLockTimer -= dt;

  processInputQueue(dt);

  for (let bed of beds) {
    bed.update(dt);
  }

  updateSurge(dt);
  updateTension(dt);
  updateCombo(dt);
  updateScore(dt);
  updateParticles(dt);
  checkEndConditions();
}

// ==============================================================
// INPUT HANDLING
// ==============================================================
function keyPressed() {
  if (key === 'v' || key === 'V') {
    reducedEffects = !reducedEffects;
    return;
  }
```

```
  switch (gameState) {
    case STATE.TITLE:
      if (keyCode === ENTER || key === ' ') {
        initGame();
        gameState = STATE.PLAYING;
      }
      if (key === 'i' || key === 'I') {
        gameState = STATE.INSTRUCTIONS;
      }
      break;
    case STATE.INSTRUCTIONS:
      if (keyCode === ESCAPE || keyCode === BACKSPACE || key === 'i' || key === 'I') {
        gameState = STATE.TITLE;
      }
      break;
    case STATE.PLAYING:
      handlePlayingInput();
      break;
    case STATE.GROUNDING:
      if (key === ' ') {
        groundingInput();
      }
      break;
    case STATE.WIN:
    case STATE.LOSE:
      if (keyCode === ENTER || key === ' ') {
        actionBtns = [];
        initGame();
        gameState = STATE.PLAYING;
      }
      if (keyCode === ESCAPE) {
        actionBtns = [];
        gameState = STATE.TITLE;
      }
      break;
    case STATE.PAUSED:
      if (key === 'p' || key === 'P') {
        gameState = STATE.PLAYING;
      }
      break;
  }
}

function handlePlayingInput() {
  // Movement still works during action lock (CHANGE #4)
  let col = selectedBed % GRID_COLS;
  let row = floor(selectedBed / GRID_COLS);

  if (key === 'w' || key === 'W' || keyCode === UP_ARROW) row = max(0, row - 1);
  if (key === 's' || key === 'S' || keyCode === DOWN_ARROW) row = min(GRID_ROWS - 1, row + 1);
  if (key === 'a' || key === 'A' || keyCode === LEFT_ARROW) col = max(0, col - 1);
  if (key === 'd' || key === 'D' || keyCode === RIGHT_ARROW) col = min(GRID_COLS - 1, col + 1);
  selectedBed = row * GRID_COLS + col;

  // Actions (will be blocked inside doAction/startGrounding if locked)
```

```
  if (key === 'q' || key === 'Q') queueAction(() => doAction('water'));
  if (key === 'e' || key === 'E') queueAction(() => doAction('light'));
  if (key === 'r' || key === 'R') queueAction(() => doAction('airflow'));
  if (key === ' ') startGrounding();
  if (key === 'p' || key === 'P') gameState = STATE.PAUSED;
}

function mousePressed() {
  for (let btn of menuButtons) {
    if (btn.checkClick(mouseX, mouseY)) return;
  }

  if (gameState === STATE.PLAYING) {
    for (let btn of actionBtns) {
      if (btn.checkClick(mouseX, mouseY)) return;
    }
    let idx = getBedAtMouse(mouseX, mouseY);
    if (idx >= 0) selectedBed = idx;
  }
}

if (typeof window !== 'undefined') {
  window.addEventListener('keydown', function(e) {
    if ([32, 37, 38, 39, 40].includes(e.keyCode)) {
      e.preventDefault();
    }
  });
}
```

Summary of all changes

Change #1 — Plant images:

- Added `preload()` to load 4 PNGs from `assets/` folder
- Added `checkImagesLoaded()` fallback detection — if any image fails, old rectangles/icons are used (no crash)
- `drawBed()` now draws the correct sprite based on health thresholds (67+/34+/1+/0), with a reddish tint overlay for dead plants
- All UI overlays (bars, alerts, borders, uncertain zone) draw on top of the image

Change #2 — Buffed airflow:

- `AIRFLOW_DURATION`: 5 → **8** seconds (+60%)
- `AIRFLOW_DRAIN_REDUCTION`: 0.5 → **0.3** (slows drain to 30% instead of 50%)
- `AIRFLOW_COOLDOWN`: 4 → **3** seconds
- NEW: `AIRFLOW_HEALTH_RECOVERY = 1.5` — passive healing while airflow is active
- NEW: Airflow halves health loss rate on stressed plants

Change #3 — Easier grounding:

- `GROUNDING_BEAT_WINDOW`: 0.3 → **0.45** seconds (50% wider)
- `GROUNDING_BEAT_INTERVAL`: 1.2 → **1.5** seconds (slower rhythm)
- NEW: `GROUNDING_SUCCESS_THRESHOLD = 1` — only need **1 of 3** hits to succeed
- NEW: "Perfect!" feedback for tight timing, plus hit counter display
- Updated instruction text: "Press when the circle is largest" + "(generous timing)" hint

Change #4 — Action lock (anti-spam):

- NEW: `ACTION_LOCK_DURATION = 0.2` seconds
- `actionLockTimer` blocks water/light/airflow/grounding while active
- Triggered after water or light actions (not airflow — it already has a long cooldown)
- Movement still works during lock
- Buttons dim during lock for visual feedback, plus subtle board overlay

Constants to tweak for balance (all at the top of `sketch.js`):

- `ACTION_LOCK_DURATION` — increase to 0.3 if still too spammable
- `AIRFLOW_HEALTH_RECOVERY` — increase for stronger healing
- `GROUNDING_SUCCESS_THRESHOLD` — set to 2 for harder grounding
- `GROUNDING_BEAT_WINDOW` — decrease toward 0.3 for tighter timing
- `PLANT_IMG_GOOD_MIN` / `PLANT_IMG_OKAY_MIN` — adjust which image shows at what health

Files hidden in shared chats

are the photos on top? i want them to fill in the boxes and i want the plants to be seen, heres the photo for example, i want the water light and health bar to be at the bottom of the square where theres just soil so they dont cover the plant

Mar 11

Got it — the image should fill the whole bed as a background, and the bars should sit at the bottom over the soil area. Here's the updated `drawBed()` function — replace the entire existing one:

js
```
// ============================================================
// CHANGE #1 — Rewritten drawBed(): image fills box, bars at bottom
// ============================================================
function drawBed(bed) {
  let bx = bed.x;
  let by = bed.y;
  let bw = bed.w;
  let bh = bed.h;

  // --- Background rectangle (base, visible if no image) ---
  if (bed.isWilted) {
    fill(COL.bedDead[0], COL.bedDead[1], COL.bedDead[2]);
```

```
} else if (bed.isUncertainZone) {
  let shimmer = sin(millis() / 300 + bed.index) * 10;
  fill(COL.bedUncertain[0] + shimmer, COL.bedUncertain[1] + shimmer, COL.bedUncertain[2] + shimmer);
} else {
  fill(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]);
}

if (bed.flashTimer > 0) {
  let flashAlpha = bed.flashTimer / 0.3;
  fill(lerpColor(
    color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
    color(COL.accent[0], COL.accent[1], COL.accent[2]),
    flashAlpha * 0.3
  ));
}

if (bed.restoreFlash > 0) {
  let flashAlpha = bed.restoreFlash / 0.5;
  fill(lerpColor(
    color(COL.bedNormal[0], COL.bedNormal[1], COL.bedNormal[2]),
    color(100, 255, 150),
    flashAlpha * 0.4
  ));
}

stroke(60, 70, 60);
strokeWeight(1);
rect(bx, by, bw, bh, 4);
noStroke();

// --- Draw plant image filling the entire bed box ---
if (imagesLoaded) {
  let img = getPlantImage(bed.health);
  if (img) {
    push();
    imageMode(CORNER);
    // Clip to the bed rectangle (draw image filling entire box)
    // Dead plants get a dark desaturated tint
    if (bed.isWilted) {
      tint(80, 50, 50, 200);
    }
    image(img, bx + 1, by + 1, bw - 2, bh - 2);
    noTint();
    pop();
  }
} else {
  // Fallback: old text icon rendering if images didn't load
  if (bed.isWilted) {
    fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 120);
    textAlign(CENTER, CENTER);
    textSize(28);
    text('✕', bx + bw / 2, by + bh / 2 - 10);
    textSize(12);
    text('Wilted', bx + bw / 2, by + bh / 2 + 18);
    return;
```

```
  }

  let healthColor = bed.health > 50 ?
    COL.health :
    [lerp(COL.healthLow[0], COL.health[0], bed.health / 50),
     lerp(COL.healthLow[1], COL.health[1], bed.health / 50),
     lerp(COL.healthLow[2], COL.health[2], bed.health / 50)];

  fill(healthColor[0], healthColor[1], healthColor[2]);
  textAlign(CENTER, CENTER);
  textSize(22);
  let plantIcon = bed.health > 60 ? '❋' : bed.health > 30 ? '❊' : '✿';
  text(plantIcon, bx + bw / 2, by + bh * 0.35);
}

// If wilted, show label and skip bars
if (bed.isWilted) {
  // Dark overlay so "Wilted" text is readable over the image
  fill(0, 0, 0, 120);
  noStroke();
  rect(bx + 1, by + bh - 30, bw - 2, 29, 0, 0, 4, 4);
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 220);
  textAlign(CENTER, CENTER);
  textSize(13);
  textStyle(BOLD);
  text('Wilted', bx + bw / 2, by + bh - 16);
  textStyle(NORMAL);
  return;
}

// ============================================================
// BARS AT THE BOTTOM — positioned over the soil area
// so they don't cover the plant in the upper portion
// ============================================================
let barH = 6;
let barW = bw - 16;
let barX = bx + 8;
let barGap = 4;      // gap between label and next bar
let labelH = 10;     // height for tiny label text

// Calculate bar area from bottom up
// Layout from bottom: [light label] [light bar] [water label] [water bar] [hp label] [hp bar] [padding]
let bottomPad = 6;
let totalBarArea = 3 * (barH + labelH + barGap);
let barAreaTop = by + bh - bottomPad - totalBarArea;

// Semi-transparent dark backdrop behind bars so they're readable over soil
fill(0, 0, 0, 140);
noStroke();
rect(bx + 1, barAreaTop - 4, bw - 2, bh - (barAreaTop - by) + 3, 0, 0, 4, 4);

let healthColor = bed.health > 50 ?
  COL.health :
  [lerp(COL.healthLow[0], COL.health[0], bed.health / 50),
   lerp(COL.healthLow[1], COL.health[1], bed.health / 50),
```

```
  lerp(COL.healthLow[2], COL.health[2], bed.health / 50)];

let curY = barAreaTop;

// --- Health bar ---
textSize(8);
textStyle(BOLD);
fill(healthColor[0], healthColor[1], healthColor[2]);
textAlign(LEFT, TOP);
text('HP ' + floor(bed.health), barX, curY);
textStyle(NORMAL);
curY += labelH;

fill(20, 25, 20);
rect(barX, curY, barW, barH, 2);
fill(healthColor[0], healthColor[1], healthColor[2]);
rect(barX, curY, barW * bed.health / 100, barH, 2);
curY += barH + barGap;

// --- Water bar ---
textSize(8);
fill(COL.water[0], COL.water[1], COL.water[2]);
textAlign(LEFT, TOP);
text('💧 ' + floor(bed.water), barX, curY);
curY += labelH;

fill(20, 25, 35);
rect(barX, curY, barW, barH, 2);
fill(COL.water[0], COL.water[1], COL.water[2]);
rect(barX, curY, barW * bed.water / 100, barH, 2);
curY += barH + barGap;

// --- Light bar ---
textSize(8);
fill(COL.light[0], COL.light[1], COL.light[2]);
textAlign(LEFT, TOP);
text('☀ ' + floor(bed.light), barX, curY);

// Airflow indicator next to light label
if (bed.airflowActive) {
  fill(COL.airflow[0], COL.airflow[1], COL.airflow[2], 200);
  textAlign(RIGHT, TOP);
  text('🌬' + floor(bed.airflowTimer) + 's', barX + barW, curY);
  textAlign(LEFT, TOP);
}
curY += labelH;

fill(20, 25, 20);
rect(barX, curY, barW, barH, 2);
fill(COL.light[0], COL.light[1], COL.light[2]);
rect(barX, curY, barW * bed.light / 100, barH, 2);

// ============================================================
// STATUS OVERLAYS — drawn on top of everything
// ============================================================
```

```
  // Surge flicker overlay
  if (surgeActive && !reducedEffects && random() < 0.15) {
    fill(COL.surge[0], COL.surge[1], COL.surge[2], 50);
    noStroke();
    rect(bx, by, bw, bh, 4);
  }

  // False alert / urgency indicators (top-right corner)
  if (bed.hasFalseAlert && !bed.isWilted) {
    fill(COL.falseAlert[0], COL.falseAlert[1], COL.falseAlert[2], 160 + sin(millis() / 150) * 60);
    textAlign(RIGHT, TOP);
    textSize(16);
    text('⚠', bx + bw - 4, by + 4);
  } else if (bed.displayUrgency === 'critical') {
    fill(COL.healthLow[0], COL.healthLow[1], COL.healthLow[2], 180);
    textAlign(RIGHT, TOP);
    textSize(14);
    text('!!', bx + bw - 6, by + 4);
  } else if (bed.displayUrgency === 'warning') {
    fill(COL.light[0], COL.light[1], COL.light[2], 140);
    textAlign(RIGHT, TOP);
    textSize(13);
    text('!', bx + bw - 6, by + 6);
  }

  // Uncertain zone border + label
  if (bed.isUncertainZone) {
    noFill();
    stroke(COL.bedUncertain[0] + 40, COL.bedUncertain[1] + 40, COL.bedUncertain[2] + 40, 100 + sin(millis() /
400 + bed.index) * 50);
    strokeWeight(2);
    rect(bx + 2, by + 2, bw - 4, bh - 4, 3);
    noStroke();

    // Label in top-left area (over the plant, not the bars)
    fill(180, 160, 220, 180);
    textAlign(LEFT, TOP);
    textSize(8);
    text('uncertain', bx + 5, by + 5);
  }

  // Bed index (small debug label, top-left)
  fill(COL.textSecondary[0], COL.textSecondary[1], COL.textSecondary[2], 80);
  textAlign(LEFT, TOP);
  textSize(8);
  text('#' + (bed.index + 1), bx + 4, by + (bed.isUncertainZone ? 15 : 4));
}
```

What changed

The image now fills the **entire bed box** edge to edge using `image(img, bx+1, by+1, bw-2, bh-2)`. The HP, water, and light bars are positioned at the **bottom ~40%** of the box (over the soil

in your artwork), with a semi-transparent dark backdrop so they're readable against the soil texture. The plant in the upper portion stays fully visible and unobstructed.

The bar area auto-calculates from the bottom up, so if you ever change bed sizes, the bars will stay anchored to the bottom. Alert icons and uncertain zone labels sit in the top corners where there's leafy background, not over the bars.

whats the canvas size?